XLOG

A CUDA-Native Engine for Neurosymbolic Integration

Levi Dubrovin, Nikita Pospelov, Kirill Sabitov

Brainyblaze Dynamics Inc.

levi@brainyblaze.com, nikita_pospelov@brainyblaze.com, kirill@brainyblaze.com

https://github.com/BrainyBlaze/xlog

Technical Whitepaper

July 2026

Abstract

xlog is a CUDA-native logic programming engine that couples neural perception with deterministic Datalog, probabilistic inference, and epistemic reasoning over world views through a shared typed frontend and provider-owned CUDA runtime. The routes share device data planes but not one universal execution boundary: ordinary Datalog and exact inference are host-orchestrated, while the certified resident recursive and Monte Carlo sampled cores record zero tracked host–device transfers within their measured regions. The recursive graph then returns one pinned terminal receipt; Monte Carlo reads three postflight metadata words, with result reads separate. End-to-end gradients are realized through the probabilistic path. After host provenance construction and upload, a CUDA-backed knowledge-compilation pipeline (CNF → Decision-DNNF → circuit) turns the formula into a differentiable circuit whose forward pass is exact weighted model counting and whose backward pass yields exact gradients. The final smoothed circuit is certified against its back-end source formula before it is cached or evaluated: bounded status and error scalars are observed by the host, while resolution-proof traces are checked on the GPU. A resident Monte Carlo alternative covers supported programs for which approximate inference is selected. Circuit caching yields a 2.74× MNIST-addition training speedup, and a worst-case-optimal join subsystem a 27.96× geometric-mean gain over xlog's own binary-join baseline. Against external engines the picture is mixed: MNIST-addition accuracy matches Scallop's (0.9561 against 0.9468) and we make no per-epoch speed claim, because the baseline's epoch time is not monotone in the host's CPU quota; across five hub-skewed triangle-counting cases the Soufflé-to-fused-xlog execution-time ratio *grows with the input*, from 0.88× at 150k edges — where Soufflé is the faster engine — to 5.54× at 1.2M, while fused peak device allocations stay between 85 and 1,033 MB against 3,287 to 44,979 MB for the materializing arm, and every arm completes; and exact inference is correctness-equivalent to but slower than ProbLog2. On a public video benchmark, a proximity predicate trained through symbolic credit alone, never shown a distance label, replaces hand-set geometry at no cost in held-out accuracy; the same detector inside the Event-Calculus rule search does not survive ten-fold cross-validation, and on a leak-free split the committed clause does not transfer. On a maritime corpus of thousands of gold events, weighted clauses beat crisp selection by 0.065 F1, and a single chronological training pass reproduces that result exactly.

1 Introduction

Neurosymbolic systems combine neural perception with symbolic reasoning, but established integrations often place the two halves on different processors. Deep learning frameworks—PyTorch, JAX—execute dense tensor computation on the GPU through optimized CUDA kernels, while systems such as DeepProbLog [1] and NeurASP [2] use CPU-side symbolic backends. Their integration can therefore cross the PCIe bus for network outputs, symbolic results, or gradients. At scale, repeated host–device movement can dominate wall-clock time and bandwidth.

The transfer wall is one half of the problem; fragmented interfaces are the other. Existing neurosymbolic frameworks commonly bridge a neural network to one symbolic backend (DeepProbLog to probabilistic logic, NeurASP to answer-set programming). GPU logic efforts, conversely, accelerate focused workloads such as deterministic Datalog [3, 4] or SAT [5]. xlog explores how a typed language and shared CUDA services can support deterministic, probabilistic, epistemic, and neural-symbolic use without pretending that those routes have identical execution or differentiability properties.

xlog is a CUDA-native logic programming language whose compiler and runtime treat a neural network as an ordinary predicate. Network outputs can become weighted facts for probabilistic knowledge compilation, where exact query gradients flow back to the network. Other reasoning modes reuse the typed relational and solver interfaces without claiming the same gradient semantics. Device-resident buffers carry facts, deltas, circuits, solver state, and gradients, while the host performs orchestration, I/O, compilation, and bounded control

reads. The certified resident recursive and Monte Carlo sampled cores provide the stronger measured boundary: no tracked host–device transfers inside the named measured region. The recursive graph then returns one pinned receipt; Monte Carlo reads a separate three-word post-flight record and leaves result reads explicit. Verified knowledge compilation certifies the final smoothed circuit against its back-end formula before use, and DLPack [6] exports device gradient buffers directly into PyTorch's autograd graph.

The principal contributions of this paper are:

- **A shared neurosymbolic integration model.** A neural network is a predicate, and typed device-buffer interfaces connect it to the supported reasoning routes. End-to-end gradient training is realized specifically through the probabilistic knowledge-compilation path (Sections 2 and 6).
- **CUDA relational execution with a certified resident route.** Semi-naive Datalog evaluation with stratified negation and aggregation uses custom CUDA relational kernels. Eligible recursive query plans are selected automatically for a resident conditional graph whose measured core has no tracked transfers; the wider ordinary executor remains host-orchestrated. A worst-case-optimal join (WCOJ) subsystem avoids the intermediate-relation blowup of binary-join plans on skewed cyclic queries (a measured 27.96× geometric-mean ablation, Section 4).
- **Verified knowledge compilation.** A host provenance graph is uploaded into a CUDA-backed CNF → Decision-DNNF → circuit pipeline that certifies the final smoothed circuit against its back-end source formula before caching or evaluation. The host observes bounded solver status and error scalars; resolution-proof traces are checked on the GPU. Structural invariants required for weighted model counting are established separately by construction (Section 5).
- **End-to-end differentiable training with zero-copy interop.** Compiled circuits are cached across iterations and evaluated as level-parallel forward/backward kernels, with gradients exported to PyTorch through DLPack; circuit caching yields a measured 2.74× training speedup. Term embeddings use PyTorch lookups, while differentiable ILP uses a distinct relational/tagged-credit path (Section 6).
- **Perception learned through symbolic credit on an external benchmark.** On the CAVIAR video-event corpus, under a direct per-timestep target, the rule search returns the same theory whether its proximity predicate is the hand-set geometric one that dedicated rule learners are handed or a network trained by the logic's own credit alone, never shown a distance label or any target of its own, at no cost in held-out accuracy. Event-Calculus rules over the precomputed predicate are induced under pre-registered gates; substituting the learned detector inside *that* search is where the result currently fails (Section 7).
- **Rule induction at scale.** On the Brest AIS maritime corpus (3,548 gold `rendezVous` intervals) the same protocol, over pair-level folds and permutation-null gates, measures in isolation the two mechanisms the published Event-Calculus learners are built on. Weighting the clauses of the same gated pool raises micro F1 by 0.065 over crisp selection, against our own prior expectation that it would not, and learning those weights in a single chronological pass reproduces the batch predictions exactly, for a reason we can name. We draw no comparison with the published maritime figure, whose temporal grid differs from ours (Section 8).
- **Reasoning beyond probability.** Epistemic reasoning over world views (`know`/`possible` under founded and Gelfond-style semantics) execute over finite supported programs, including positive modal fixpoints and supported epistemic recursion through negation evaluated by GPU-backed well-founded semantics, reusing the CDCL, join, and circuit machinery instead of a separate engine (Section 9). The accepted fragment and its bounds are in Section 12.

Target workloads. Avoiding repeated data-plane copies matters most where the symbolic workload is large and repeated: program-analysis queries over millions of ground atoms, probabilistic logic with many training iterations that share one compiled circuit, and knowledge-graph reasoning with trainable entity embeddings. Small tasks such as MNIST addition exercise the integration end-to-end but sit below this regime. They demonstrate correctness and the circuit-caching effect, with the measured transfer component secondary: approximately 10% of a training step at a standard batch and lost in run-to-run variation when a step makes only a handful of neural→symbolic handoffs (Section 10.6). Accordingly, xlog targets NVIDIA-GPU workloads whose byte working set and fixed capacities fit the selected route (Section 12) and that need neural perception coupled to exact, verified probabilistic inference rather than fuzzy relaxations.

Section 2 presents the integration model and system architecture. Section 3 describes the xlog language. Section 4 details the GPU-resident symbolic substrate. Section 5 presents verified knowledge compilation, the central technical result. Section 6 ties the pieces together as end-to-end neurosymbolic learning. Section 7 puts that surface on an external video-event benchmark, and Section 8 carries the same induction protocol to a larger maritime corpus. Section 9 extends the substrate to epistemic and other reasoning modes. Section 10 reports evaluation results, Section 11 surveys related work,

Section 12 discusses limitations and future work, and Section 13 concludes.

2 Integration Model and Architecture

2.1 The integration model

In xlog a neural network *is* a predicate. A declaration binds a network to a predicate symbol; evaluating that predicate runs a forward pass, and the network's softmax over a label set can become a distribution over weighted facts. Typed device-buffer interfaces make those values available to the supported reasoning routes. Exact gradients of query values flow back through the probabilistic knowledge-compilation route; deterministic and epistemic execution reuse the integration surface without claiming that gradient semantics. Facts, circuit values, and gradient buffers remain device-resident at their data-plane handoffs, while orchestration and bounded control reads remain host responsibilities.

2.2 GPU residency model

xlog's device execution routes do not implicitly spill their data plane to a host executor. Each provider owns one device handle, stream pool, asynchronous resource, runtime, and memory manager. Configured byte budgets, fixed capacities, and iteration bounds remain separate limits; their exact diagnostics depend on the entry point rather than one universal status vocabulary. Transfer counters delimit measured claims. The resident recursive graph records zero tracked transfers inside its measured core and then reads one pinned terminal receipt. Resident Monte Carlo measures its sampled core separately, then reads three postflight metadata words; requested count results are read afterward, not bundled into that receipt.

The payoff is bandwidth asymmetry. PCIe 4.0 ×16 delivers roughly 32 GB/s, while GPU HBM provides 1–3 TB/s. A single unnecessary D2H round-trip for a moderate relation (10M tuples at 16 bytes, ≈160 MB) costs about 5 ms. Keeping fact stores, deltas, circuit values, solver state, and gradient buffers on the GPU avoids copying those data planes through host memory; this claim does not erase the bounded control observations documented for each route.

2.3 Crate architecture

Figure 1 groups the core crates into five architectural roles; their Cargo manifests, rather than these groups, define the dependency graph. In particular, `xlog-solve` depends on `xlog-runtime`. A leaf *core* crate defines shared scalar types, error and memory-budget types, canonical boolean parsing, and a bidirectional symbol table for dictionary-encoded strings. Intermediate representations and device services wrap CUDA via cudarc [7], resolve embedded kernel modules, and expose Arrow and DLPack interoperability. Each CUDA provider owns its complete runtime resource graph; there is no process-wide runtime singleton or alternate provider path. The typed language frontend of Section 3, GPU runtime executor, and GPU CDCL/local-search solver compose into deterministic and probabilistic pipelines. Neural integration adds a CUDA-independent registration/bridge crate with optional Python support; the Python extension connects it to those execution pipelines. A PyO3 [8] extension module (`pyxlog`) and command-line binary surface the supported routes to users.

2.4 From program to GPU plan

A program is compiled to a GPU-executable plan in five stages.

Parsing. A PEG parser built with pest produces an AST covering the full surface, including probabilistic facts, annotated disjunctions, epistemic literals, and neural-predicate declarations.

Dependency analysis builds a graph with positive, negative, aggregate, probabilistic, and modal edges and computes its strongly connected components (SCCs) with Tarjan's algorithm. It establishes a deterministic stratum order for the ordinary monotone core and classifies supported nonmonotone components for their dedicated plans.

Lowering fans out from the normalized AST into three reasoning IRs: deterministic and reduced ordinary rules become relational-IR trees, probabilistic provenance becomes PIR, and modal programs retain their semantics in EIR. Neural predicates compose with the applicable route. SAT/MaxSAT and verification use a separate shared service instead of a fourth reasoning IR.

Optimization pushes predicates through joins, while lowering chooses a deterministic greedy join order. Shape-specific selectivity rewrites cover recognized triangle and four-cycle plans; general dynamic-programming join ordering is not active. Eligible multiway rules are promoted to the WCOJ subsystem, and bound recursive queries can be rewritten via magic sets (Section 4).

Plan assembly emits the route-specific execution plan, whose relational work is dispatched to CUDA kernels. Supported negated epistemic recursion uses GPU-backed well-founded semantics instead of the ordinary semi-naive schedule; probabilistic provenance analysis has a separate host-side well-founded evaluator.

2.5 Reasoning IRs and circuit format

Three reasoning IRs keep the backends decoupled; probabilistic knowledge compilation also emits a downstream circuit format. Each form has one role.

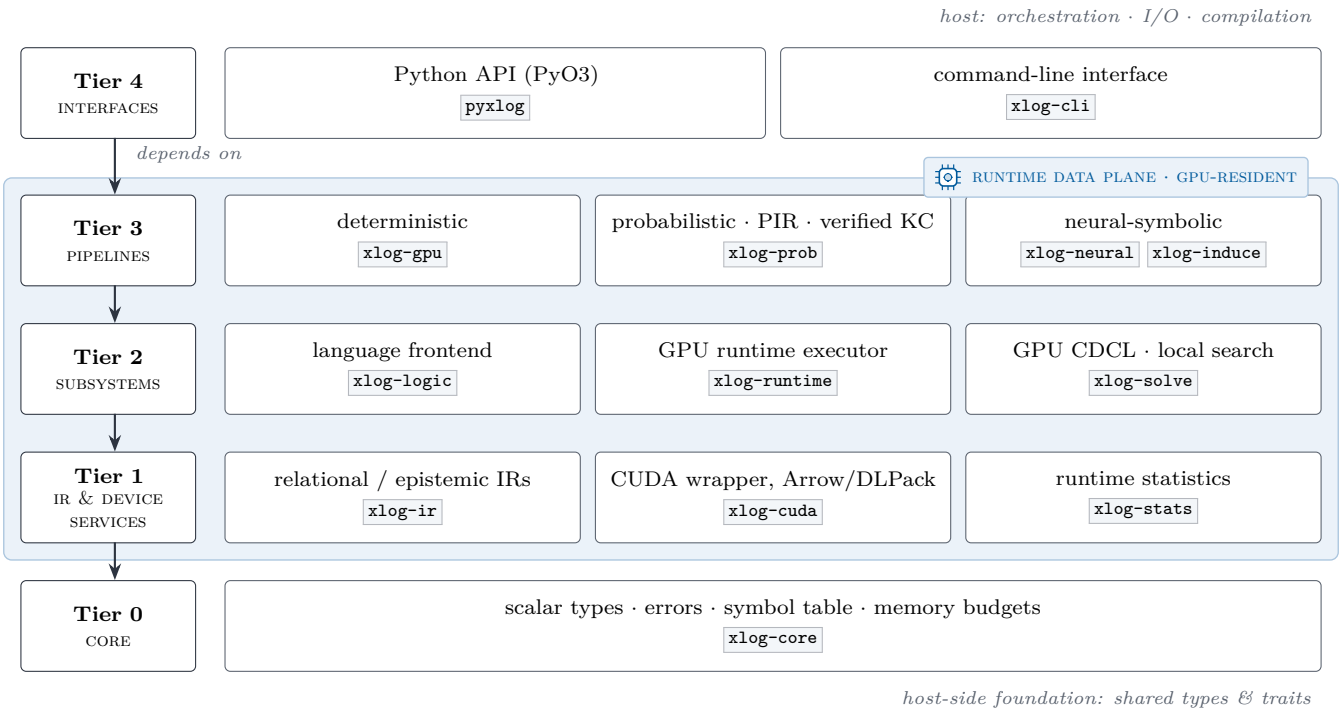


Figure 1: xlog crates grouped by architectural role. These groups are not strict Cargo dependency levels: the solver depends on the runtime, and the runtime can enable the language frontend for selected tests. The shaded plane marks GPU-resident runtime data, with host orchestration and control reads.

RIR (Relational IR) is the algebraic tree (`Scan`, `Filter`, `Project`, `Join`, `GroupBy`, `Union`, `Distinct`, `Diff`, `Fixpoint`, and the worst-case-optimal `MultiWayJoin`/`ChainJoin`) carrying cardinality, skew, and delta-vs-full metadata. Deterministic execution and reduced ordinary subprograms consume RIR; the other backends preserve additional semantics in their own IRs.

PIR (Provenance IR) is the weighted Boolean formula (`Lit`, `NegLit`, `And`, `Or`, `Decision`) derived from provenance tracking over a probabilistic program, capturing its structure without execution order.

XGCF (xlog GPU Circuit Format) is the compiled, device-resident form: a levelized DAG whose level offsets let every node at one topological level evaluate in a single kernel launch, with forward (log-space weighted model counting) and backward (adjoint) passes.

EIR (Epistemic IR) preserves modal operators for world-view reasoning and lowers to a dedicated GPU execution plan instead of ordinary relational algebra (Section 9).

GpuCnf and CDCL form a shared solver service, not a fourth reasoning IR. Callers provide a device-resident CNF formula for SAT/MaxSAT search or verification. The CDCL service consumes that formula directly; XGCF remains the circuit format produced only by probabilistic knowledge compilation.

3 The xlog Language

Probabilistic facts (`p::f.`) and neural predicate declarations (`nn/k`) are language-level constructs covered in Sections 5 and 6.

3.1 Design principles

Three principles shape the surface, each chosen to enable GPU execution. **Typed predicates:** every predicate has a statically known signature over a closed set of scalar types (`u32`, `u64`, `i32`, `i64`, `f32`, `f64`, `bool`, `symbol`), supplied by a `pred` declaration or inferred during compilation, so argument mismatches are caught before any kernel is generated and allocation stays bounded and columnar. **Classified dependencies:** negation, aggregation, modal operators, and recursion interact through SCC analysis on the predicate dependency graph. The ordinary deterministic core requires a monotone or stratified schedule, while supported nonmonotone probabilistic and epistemic components enter dedicated plans, including GPU-backed epistemic well-founded semantics and a separate host-side well-founded provenance analysis for supported probabilistic recursion through negation. **One language for many paradigms:** probabilistic facts, annotated disjunctions, neural predicates, and SAT constraints share the syntactic core of deterministic Datalog; parsing, normalization, and dependency analysis are shared, while backend-specific lowering preserves the

semantics needed by each plan. Unsupported shapes and unbounded terms are rejected before execution; there is no implicit fallback.

A minimal program exhibits the core surface:

```
// Typed predicates, facts, a rule, a query.
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :- edge(X, Y).
reach(X, Z) :- reach(X, Y), edge(Y, Z).

?- reach(1, N).
```

Listing 1: Minimal xlog program: typed predicates, facts, a recursive rule, a query.

3.2 Type system

Every predicate is assigned a schema over the eight scalar types, with `f32`/`f64` following IEEE 754. A `pred` declaration supplies that schema explicitly; when it is omitted, compilation infers the schema from facts, rule heads, typed body columns, and arithmetic bindings. String literals (`"alice"`) map to `symbol` through a global symbol table, giving the runtime dense integer identifiers while preserving source-level readability; `domain` declarations introduce named aliases. Type consistency is enforced by predicate schemas and lowering-time checks: variable occurrences must agree with known column types, constants are validated against their destination columns, and arithmetic expressions preserve scalar types.

```
pred node(u32, symbol).
pred connected(u32, u32).
pred bridge(u32, u32).

node(1, "alice"). node(2, "bob").
connected(1, 2).

bridge(A, B) :-
  connected(A, B),
  node(A, _), node(B, _).
```

Listing 2: A well-typed xlog program: typed bridge predicate.

Declaring `bridge` so that its head argument is constrained at two incompatible types (e.g. `symbol` in the head but `u32` through `connected`) rejects the program at compile time.

3.3 Expressions, arithmetic, and user-defined functions

Rule bodies may contain arithmetic, comparisons, and calls. Arithmetic bindings use the `is` operator (`D is X + Y`); the right-hand side supports `+ - * / %`, the built-ins `abs`, `min`, `max`, `pow`, `cast`, and user-defined functions. Comparisons compile to `Filter` nodes and push below joins where profitable. Float comparison uses a total ordering, so `NaN` and signed zeros yield deterministic results instead of contaminating downstream aggregations.

```
pred point(u32, f64, f64).
pred close(u32, u32).

point(1, 0.0, 0.0).
point(2, 0.5, 0.5).
point(3, 5.0, 5.0).

close(A, B) :-
  point(A, Xa, Ya),
  point(B, Xb, Yb),
  A < B,
  D is (Xa - Xb) * (Xa - Xb)
     + (Ya - Yb) * (Ya - Yb),
  D < 1.0.
```

Listing 3: Arithmetic in rule bodies: pairwise proximity via squared Euclidean distance.

A user-defined function (UDF) is introduced with `func`, with optional parameter types and a return type after `->`. Arithmetic bodies (optionally with `if/then/else`) inline into the same `Expr` trees as built-in arithmetic and participate in the same predicate-pushdown and expression-level optimization. A predicate body, as in `func get_parent(Child) = Parent :- parent(Child, Parent).`, contributes its relational literals immediately before the calling `is` binding in an ordinary rule or integrity constraint. Its relation may contribute zero, one, or many caller rows; the function syntax does not impose uniqueness, while ordinary set semantics deduplicates identical projected tuples. Each invocation alpha-renames its non-parameter result and body-local variables, and multiple calls expand from left to right. Nested expansion is depth-bounded; relational calls in conditional result branches and non-term arithmetic arguments are rejected rather than hoisted or coerced with changed semantics.

```
pred raw_score(u32, f64).
pred norm_score(u32, f64).

func clamp01(X: f64) -> f64 =
  if X < 0.0 then 0.0
  else if X > 1.0 then 1.0
  else X.

raw_score(1, -0.3).
raw_score(2, 0.7).
raw_score(3, 1.4).

norm_score(Id, N) :-
  raw_score(Id, R), N is clamp01(R).
```

Listing 4: User-defined function: `clamp01` normalizes a raw score into the unit interval.

3.4 Modules and imports

Programs decompose into `.xlog` modules on a search path. The `use` directive loads a module (`use graph.`) or selectively imports names (`use utils/math::{abs, clamp}.`); a `private` modifier hides declarations from importers. The resolver traverses the import graph, detects cycles, validates participating declarations, and performs bounded cross-module schema inference for undeclared predicate contributions before merging them. The resulting logical program then undergoes full compiler type inference, dependency analysis, and route-specific execution planning, so imported rules participate as if written inline.

```
// library: graph_lib.xlog
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :- edge(X, Y).
reach(X, Z) :- reach(X, Y), edge(Y, Z).
```

Listing 5: Module `graph_lib.xlog`: predicate declarations and recursive reach rules.

```
// entry: graph_main.xlog
use graph_lib.

?- reach(1, N).
```

Listing 6: Entry `graph_main.xlog`: imports `graph_lib` and queries `reach`.

3.5 Aggregations and constraints

Aggregation operators—`count`, `sum`, `min`, `max`, `logsumexp`—appear at rule heads with an explicit aggregation variable and lower to `GroupBy` nodes executed as radix-sort-and-reduce kernels; the stratifier rejects aggregation inside a recursive SCC. Integrity constraints are headless rules (`:- body.`) asserting that `body` is unsatisfiable once evaluation reaches fixpoint; a violation raises a diagnostic. `logsumexp` is included because it is the workhorse aggregation for probabilistic circuits, where log-space numerical stability is essential (Section 5).

```
pred edge(u32, u32).
pred degree(u32, u32).

edge(1, 2). edge(1, 3).
edge(1, 4). edge(2, 3).

degree(X, count(Y)) :- edge(X, Y).

:- edge(X, _), not degree(X, _).

?- degree(X, N).
```

Listing 7: Stratified aggregation: per-source out-degree with an integrity constraint.

3.6 Completeness over finite domains

The surface extends toward language completeness while preserving GPU-native execution: accepted constructs normalize into the typed AST and lower through their semantic backend, while forms that cannot be lowered safely are rejected at compile time, since there is no hidden CPU interpreter to execute them. The accepted extensions are finite typed lists (`[]`, `[H|T]`, the `list<T>` column type), statically-resolvable meta-predicates (`ground`, `functor`, `=..`, source-fact `findall`, `maplist`), explicit stratified negation-as-failure over deterministic relations, magic-set rewriting of bound recursive queries, and finite probabilistic aggregates. Deterministic extensions lower to relational joins and aggregations in RIR; probabilistic aggregates preserve their probabilistic intermediate representation. Open-ended generators, dynamic database mutation, unrestricted `call/N`, and runtime-variable predicate names remain outside the contract.

4 GPU-Native Symbolic Substrate

The deterministic Datalog engine supplies CUDA relational operators and device-resident relation storage used across the system. The algorithmic approach—semi-naive fixpoint iteration over stratified programs—is well established; the contribution here is engineering. The ordinary executor is host-orchestrated. For eligible recursive query plans, automatic selection instead uses a certified resident conditional graph: its core runs one recorded launch and synchronization with zero tracked host–device transfers, then returns one pinned terminal receipt carrying status, counters, trace metadata, and schema selection rather than tuple data.

4.1 Semi-naive evaluation on GPU

The executor processes an execution plan as strata ordered by the dependency analysis of Section 2. Non-recursive SCCs evaluate each rule once, dispatching its

RIR tree to the appropriate kernel and merging results per head predicate. Recursive SCCs run semi-naive iteration (Algorithm 1): each rule is re-evaluated through delta-rewritten variants (one per recursive scan occurrence, so a self-join such as `p(X,Y), p(Y,Z)` contributes two variants) and the new tuples are unioned, differenced against the full relation, and deduplicated, all on-device, until no predicate produces new tuples. The profiler records per-operator timing, row counts, and peak memory, feeding the optimizer.

Algorithm 1 GPU semi-naive evaluation of a recursive SCC. All buffers are device-resident.

Require: SCC rules $\mathcal{R}$; relation store S (device-resident)
Ensure: least fixpoint merged into S, or an iteration-limit error

for each rule $r \in \mathcal{R}$ **do**
  $\Delta_r \leftarrow \textsc{Eval}(r, S) \setminus S$ ▷ *seed Δ; set difference on device*
for each head predicate p **do**
  $\Delta_p \leftarrow \textsc{Dedup}(\bigcup_{r \text{ for } p} \Delta_r)$
  $S_p \leftarrow S_p \cup \Delta_p$ ▷ *install all seeds after evaluating them*
repeat
  for each rule $r \in \mathcal{R}$, recursive scan i in r **do**
    $T_{r,i} \leftarrow \textsc{Eval}(r[\Delta \text{ at } i], S)$ ▷ *one Δ-variant per self-join*
  for each head predicate p **do**
    $\Delta_p \leftarrow \textsc{Dedup}\big(\big(\bigcup_{r,i \text{ for } p} T_{r,i}\big) \setminus S_p\big)$
    $S_p \leftarrow S_p \cup \Delta_p$ ▷ *union + sorted dedup on device*
until all $\Delta_p = \emptyset$ **or** iteration cap reached
report an error if the cap was reached before convergence
free all Δ buffers

4.2 Relational kernels

xlog implements its relational algebra as direct CUDA kernels rather than wrappers over cuBLAS, cuSPARSE, or Thrust. Bespoke kernels make allocation, synchronization, and host staging explicit and keep relational intermediate buffers on-device. Table 1 summarizes them.

The filter kernel makes a correctness-critical choice for floating point. Equality and ordering compare total-order keys derived from the exact `f32` or `f64` bits, not ordinary IEEE numerical equality. Equal NaN encodings compare equal, distinct NaN encodings remain distinct, and -0.0 differs from $+0.0$. The key transform orders negative NaNs, negative infinity, negative finite values, signed zeros, positive finite values, positive infinity, and positive NaNs deterministically.

4.3 Adaptive join planning

The optimizer performs predicate pushdown using runtime statistics, decomposing filters above joins into left-, right-, and cross-predicates. Lowering chooses join order with a deterministic greedy heuristic. A selectivity pass recognizes triangle and four-cycle shapes, and prior execution statistics can be exported and merged before a later compilation. A general dynamic-programming join ordering is not active; configuration fields that resemble such a threshold do not change this planning path.

Table 1: Core relational CUDA kernels.

Kernel	Purpose
`join`	Hash join with linked-list collision chains and FNV-1a composite hashing over raw key bytes, so all scalar types share one path; count → scan → materialize variants for inner and left-outer joins.
`sort`	Stable 4-bit radix sort (8 passes over 32-bit keys): per-pass histogram, prefix sum, scatter.
`filter`	Comparison operators for column-vs-constant and column-vs-column, with stream compaction via prefix sum.
`groupby`	Sorted-input grouped aggregation: boundary detection, then per-group Count/Sum/Min/Max/LogSumExp reduction.
`dedup`	Sort-based deduplication and set difference with type-aware equality, including IEEE 754 float handling.
`set_ops`	Union (concat + sort + dedup) and set difference via sorted-array binary search.
`scan`	Blelloch parallel prefix sum, the building block for filter compaction, dedup, and group-by.
`pack`	Multi-column key packing into row-major byte arrays with FNV-1a hashing.

4.4 Worst-case-optimal joins

Binary-join plans can materialize intermediates asymptotically larger than the final result, the classic pathology for cyclic queries such as triangle enumeration. This is not a corner case for xlog's target workloads: program-analysis queries (points-to, call-graph and reachability inference over code-dependency graphs) can contain cyclic, skewed joins where intermediate size governs cost. xlog addresses this with a worst-case-optimal join (WCOJ) subsystem [9–11]. A pure-Rust hypergraph planner analyzes rule bodies for multiway-join eligibility, derives a deterministic variable ordering from cardinality and selectivity statistics, and emits a `MultiWayJoin`/`ChainJoin` node when promotion preserves output semantics; other rules retain their ordinary binary-join plan.

The GPU kernel executes the join via count → device prefix scan → materialize over flat sorted-column storage, with a four-byte metadata D2H total and no count-vector transfer. Coverage spans triangles, 4-cycles, K-clique templates (K=5–8), recursive/SCC integration, and helper-relation splitting. Adaptive routing defaults to the cardinality cost model; the currently selectable SkewClassifier returns false for adaptive triangle and four-cycle decisions, acting as a conservative opt-out rather than a live histogram classifier. The benchmark's high-skew and uniform/empty routing outcomes therefore depend on its configured model and seeded statistics, not on a universal runtime skew-classification rule. Section 10.2 reports the retained measurements.

4.5 Runtime optimization

Interactive and long-running deployments (REPL sessions, iterative solver loops, incremental training) repeatedly evaluate similar queries over relations that change little between calls. A stateless executor rebuilds join indices and recomputes shared subplans each time. Three optimizations exploit this stability with explicit controls. Common-subexpression elimination caches safe deterministic subplans within an evaluation. Adaptive reoptimization adopts a compiler-supplied candidate plan when mis-plan telemetry crosses a configurable ratio, with output-equivalence checks and rollback. And a persistent hash-index manager, keyed on relation generation, schema, keys, and device, retains built indices across repeated sessions with deterministic LRU eviction (Section 10). These are reuse mechanisms, not a blanket zero-transfer guarantee for evaluation and diagnostics.

4.6 Reversible symbols

Datalog programs operate on strings, but GPU kernels operate on fixed-width numeric columns. xlog interns each unique string to a dense, sequential `u32` identifier and resolves it back in $O(1)$ at output time. Symbol columns then *are* ordinary `u32` columns (join, sort, filter, and dedup kernels operate on them with no special-casing) and the string table stays host-side, since variable-length data is needed only at ingestion and output. The `symbol` type maps to Arrow's `Dictionary(UInt32, Utf8)`. This preserves the compact dictionary representation; it does not make host Arrow export a zero-transfer CUDA handoff.

5 Verified Knowledge Compilation

Knowledge compilation is the route that makes a neural network and a probabilistic logic program differentiable end-to-end. A network's softmax becomes a distribution over weighted facts; provenance tracking yields a weighted Boolean formula; and that formula compiles into an arithmetic circuit whose forward pass is exact weighted model counting and whose backward pass yields exact gradients. Probabilistic systems such as ProbLog [12] follow this compile-once/evaluate-many model on the CPU. After host provenance construction and upload, xlog uses CUDA buffers for CNF encoding, circuit compilation, and evaluation while the host orchestrates stages and reads bounded control state. Before use, xlog certifies the final smoothed circuit against its back-end source formula; this certificate does not cover provenance extraction or CNF encoding.

5.1 The compilation pipeline

The public source-compilation route first extracts provenance and builds its canonical PIR graph on the host, then uploads the graph and roots. GPU CNF encoding, circuit compilation, verification, and evaluation follow under host orchestration (Figure 2). Solver completion is observed through bounded status and error scalars. This is not a host-free source-to-circuit pipeline.

Provenance extraction. Provenance over deterministic evaluation produces a weighted Boolean formula [13]. The canonical public source route builds this graph with a host-side PIR builder and relation store, then uploads it through `GpuPirGraph::from_host`. Device hash-consing primitives exist, but their availability does not make them the provenance owner on this route. Initial host construction and upload must be included when measuring source-to-result cost.

CNF encoding. A Tseitin [14] transformation lowers the formula to CNF on the GPU, using parallel prefix sums to assign compact, gap-free variable IDs and emit clauses into a device-resident compressed-sparse-row structure.

D4 compilation. The CNF compiles into a Decision-DNNF (d-DNNF) circuit [15] via a GPU-native variant of the D4 algorithm [16]: a BFS frontier exposes parallelism, per-block DFS workers perform component detection, decision-variable selection by VSIDS-like activity [17], and recursive decomposition. A smoothing pass forces every branch of an OR/decision node to mention the same variable set, and the result is levelized so each topological level evaluates in one kernel launch. Decomposability, determinism, and smoothness (the structural properties that make weighted model counting over a Decision-DNNF correct [18, 19]) are established by construction. Certification runs after smoothing and therefore gates the exact final circuit that is cached and evaluated.

5.2 Verification: a machine-checked certificate

xlog does not trust the final compilation stage; it proves the final smoothed circuit C equivalent to its back-end source formula φ (Algorithm 2). It solves $\varphi \wedge \neg C$ and $C \wedge \neg\varphi$ on a GPU CDCL solver, and both must be unsatisfiable for $C \equiv \varphi$. UNSAT results carry resolution proofs checked on-device. The host reads bounded status and error scalars before accepting that GPU proof result; a wrong, incomplete, or budget-limited result returns a typed error and never reaches circuit caching or evaluation.

The certificate says that the circuit computes the same Boolean function as φ; the count-correctness invariants are guaranteed separately, by construction. The compilation back-end thus becomes a machine-checked stage instead of an assumption, closing the failure mode in

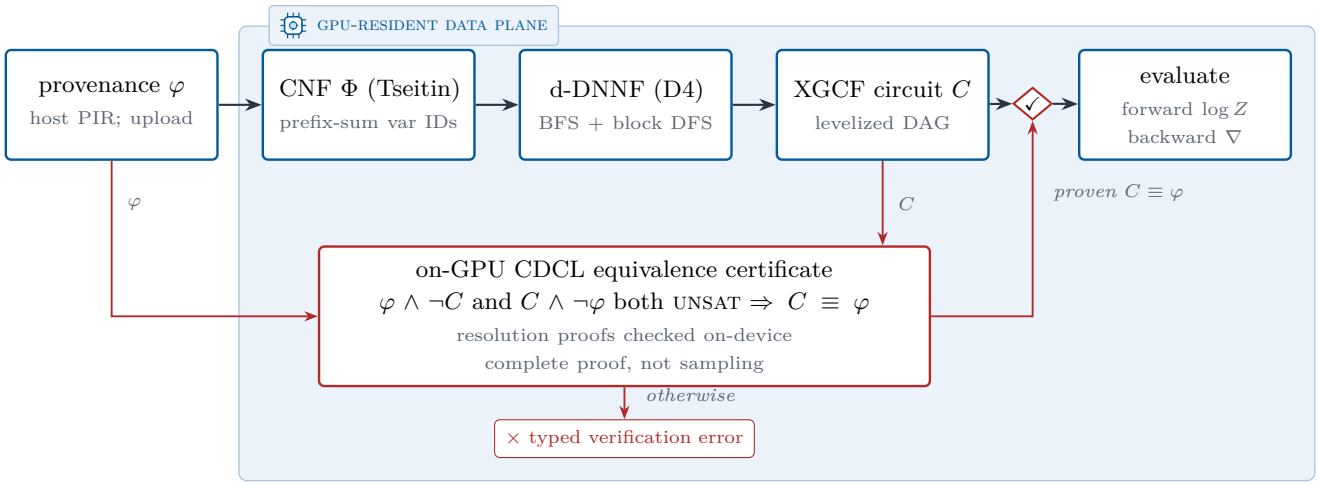


Figure 2: Verified knowledge-compilation pipeline. Host provenance extraction precedes graph upload; the shaded device data plane begins with CNF encoding. CUDA kernels operate under host orchestration. The final smoothed circuit C reaches evaluation only through the equivalence gate (✓): an on-GPU CDCL solver proves $\varphi \wedge \neg C$ and $C \wedge \neg\varphi$ both unsatisfiable; the host observes bounded status and error scalars, and any failed or incomplete proof returns a typed error.

which a silently miscompiled circuit would corrupt every downstream probability and gradient. The check covers the back-end stage specifically: provenance extraction and CNF encoding are trusted, and the certificate says nothing about them. CPU-side certified knowledge compilation exists [20]; the contribution here is performing the check on-device, sharing the GPU substrate. The same device-resident CDCL substrate is reused for SAT/-MaxSAT queries and for epistemic candidate validation (Section 9).

Algorithm 2 Verified knowledge compilation. The final smoothed circuit is accepted only if proven equivalent to its back-end source formula.

Require: canonical weighted Boolean formula φ (host PIR)
Ensure: verified circuit C (XGCF), or typed error
upload canonical PIR graph φ and selected roots to device buffers
$\Phi \leftarrow \text{TseitinCNF}(\varphi)$ ▷ *prefix-sum variable IDs; CSR clauses*
$C \leftarrow \text{D4}(\Phi)$; $C \leftarrow \text{SmoothLevelize}(C)$
$u_1 \leftarrow \text{CDCL}(\varphi \wedge \neg C)$; $u_2 \leftarrow \text{CDCL}(C \wedge \neg\varphi)$
observe bounded status and error scalars for u_1, u_2 on host
if $u_1 \neq$ UNSAT **or** $u_2 \neq$ UNSAT **then**
  return typed verification error
check resolution proofs of u_1, u_2 on-device
return C ▷ *cache the exact circuit that was certified*

The verified circuit supports a level-parallel forward pass (log-space weighted model counting, one kernel per level, yielding $\log Z$) and a reverse-order backward pass that accumulates per-variable gradients. Both leave their results in GPU-resident buffers consumable directly by PyTorch via DLPack (Section 6).

5.3 Circuit caching

Building and certifying a circuit is the most expensive stage, on the order of a minute for a cold start on the MNIST-addition benchmark, and a naive implementation pays it every epoch. Measured separately, it is the equivalence proof and not the D4 compile that dominates that time (Section 10.6). The key observation is that the circuit *structure* depends on the propositional structure of the grounded program and the evidence pattern, but *not* on the numerical weights. When neural parameters change between iterations, the weights on probabilistic facts change while the Boolean formula and its compiled circuit stay identical. The certified final smoothed circuit is therefore cached, keyed by a structural hash of the provenance graph and evidence configuration; subsequent iterations update only the weight buffers and run the forward/backward kernels. On MNIST addition that amortization is what the circuit-cache ablation of Section 10.3 measures.

5.4 Monte Carlo alternative

When the user selects approximate inference, xlog provides a GPU-parallel Monte Carlo engine for its supported positive bounded fragment, including grounded annotated disjunctions but not aggregates or negation. It samples facts and derives each world on-device, using rejection sampling or evidence clamping where supported. The resident sampled core runs in one launch and records zero tracked transfers in that measured interval. After synchronization it reads three postflight metadata words through a separately counted metadata path, not the recursive engine's pinned terminal receipt; device query/evidence counters remain available for a separate

result read. Nonconvergence returns a convergence failure rather than accepting partial counters. Structural and capacity failures return the applicable compilation or execution diagnostic. The resident entry point does not fall back to the host; a separate, explicitly enabled CPU oracle supports only its own limited fragment and is not resident GPU acceptance.

6 Neurosymbolic Integration

This section is where the relational substrate (Section 4), verified probabilistic knowledge compilation (Section 5), and device-buffer interoperability meet. Network outputs feed weighted facts into the probabilistic route; the final smoothed circuit is certified before use; and DLPack carries device gradient buffers into the neural optimizer without copying them through host memory. Host orchestration and bounded control reads remain outside that zero-copy data-plane handoff.

6.1 Neural predicates

In xlog a neural network *is* a predicate. The `nn/4` declaration embeds a network directly into the program:

```
nn(network_name, [input_vars], output_var,
   [output_labels]) :: predicate(args).
```

Listing 8: Neural predicate declaration (`nn/4`).

This is not a foreign-function escape hatch: it states that evaluating `predicate(args)` requires a forward pass through the named network, whose softmax over the label set becomes a distribution over probabilistic facts feeding the knowledge-compilation pipeline of Section 5. On the Python side a PyTorch module is registered against the predicate:

```
program.register_network("mnist_net", net, optimizer)
```

Listing 9: Registering a PyTorch module against a neural predicate.

Registration binds the declared network name to the supplied module. A caller may also state the arity of the network's predicate, and that claim is checked against every `nn/4` declaration bound to the network in the program and rejected on mismatch: the program's own declaration is the authority. Per-argument catalog sort identifiers may accompany the arity (one per declared argument, refused without an arity or at the wrong length), together with an opaque caller-supplied content hash. The engine carries those identifiers but neither derives the hash nor verifies it against model weights; re-registering a retrained network does not automatically mint a new identity. Consumers read this metadata back together with each declaration's predicate, predicate arity, input arity, and label set. A downstream rule learner can therefore check a candidate against what the program declares instead of against a naming convention—the property Section 6.5 relies on.

The dataflow runs in two directions. **Forward:** when evaluation reaches an `nn/4`-backed predicate, the registered module runs, and its softmax vector is decomposed into weighted probabilistic facts, one per label, and fed into the knowledge-compilation pipeline of Section 5, whose circuit cache carries across iterations. **Backward:** after the circuit computes the forward log-probability and backward gradients via level-parallel kernels, the per-leaf gradient buffers are exported as DLPack tensors and injected into PyTorch's autograd graph, and the optimizer updates the network as usual. The GIL is released during GPU work, so CUDA execution does not block the interpreter. A neural predicate is not confined to a query head: it may also appear inside rule bodies, so that the trainable object becomes a whole rule and not only a single perceptual predicate (Section 6.4).

6.2 End-to-end training

The canonical example is MNIST addition (the DeepProbLog benchmark): given two digit images, predict their sum, with supervision only on sums: the network never sees individual digit labels. The entire reasoning structure is two rules:

```
nn(mnist_net, [X], Y, [0,1,2,3,4,5,6,7,8,9])
    :: digit(X, Y).
addition(X, Y, Z) :-
    digit(X, D1), digit(Y, D2), Z is D1 + D2.
```

Listing 10: MNIST addition: a CNN-backed digit predicate and an addition rule.

The `nn/4` declaration connects the CNN to `digit/2`; the addition rule computes every consistent decomposition of a sum through probabilistic marginalization. Each query `addition(i, j, s)` trains by minimizing $-\log P(\texttt{addition}(i, j, s))$ under the compiled circuit. The driver compiles the program, registers the network, and runs the loop, which batches queries sharing a circuit template and runs forward/backward over the cached circuit:

```
program = pyxlog.Program.compile(SRC)
program.register_network("mnist_net", net, optimizer)
program.add_tensor_source("train", train_images)
history = pyxlog.train_model_tensor(
    program, queries,
    epochs=epochs, batch_size=batch_size)
```

Listing 11: Python driver: compile, register, train.

The decisive performance property (Figure 3) is that compilation and verification happen once, on the first

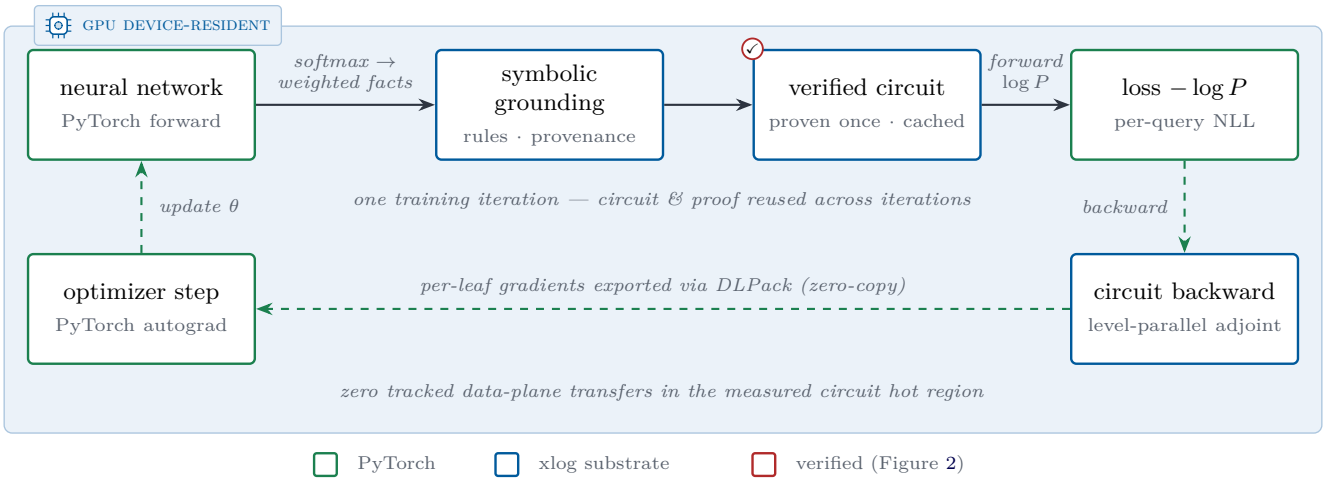


Figure 3: One neurosymbolic training iteration. Neural tensors, circuit values, and gradient buffers occupy one CUDA device (shaded plane); the measured circuit hot region records zero tracked data-plane transfers. The host still orchestrates stages and observes bounded control state. The final smoothed circuit is compiled and certified once (✓, Figure 2), then reused across iterations, and gradients re-enter PyTorch's autograd graph zero-copy through DLPack.

forward pass. Every later iteration reuses the cached circuit, executing only the weight update and the level-parallel forward/backward kernels. This is the mechanism behind the 2.74× training speedup; the loss reduction across seeds confirms that gradients flow from circuit evaluation through the logic program back to the CNN, and the full timing breakdown is reported in Section 10.

6.3 Zero-copy interoperability

A GPU-native engine is only useful if results reach the frameworks researchers already use, without copies that would reintroduce the transfer wall. xlog exposes its device-resident state through four standard interfaces.

DLPack. Each relation column or gradient buffer becomes a managed DLPack [6] tensor whose GPU memory is reference-counted and freed only when every consumer releases it; on import, xlog validates device, dtype, contiguity, and alignment, optionally against an expected schema. In Python the protocol surfaces as a capsule, so `torch.from_dlpack` yields a live CUDA tensor backed by the very memory xlog wrote.

Arrow. For columnar tools (Polars, DuckDB, cuDF), relations export through the Arrow [21] C Device Data Interface; symbol columns export as Arrow dictionary arrays, so a consumer reads symbolic columns as native string dictionaries while xlog keeps the compact integer representation.

Python bindings. The `pyxlog` extension (PyO3 [8]) exposes compilation, evaluation, training, and result extraction; tensor-valued results are returned as DLPack capsules, and long-running GPU operations release the GIL so data-loader threads proceed concurrently.

PyTorch autograd. The circuit behaves as a differentiable function inside PyTorch: the network's gradient-enabled forward feeds the circuit, the scalar loss is exported via DLPack and re-imported, and a batched path stacks inputs into one forward pass and runs a single backward, so gradients flow from the logic layer back to `nn.Module` parameters with no custom autograd `Function`.

6.4 Trainable rule bodies

Placing a neural predicate in a rule body lifts xlog from learning a single perceptual predicate to learning whole rules. The tensor and circuit data planes remain device-resident; route-specific orchestration and control observations are documented separately.

Mixed bodies. A trainable body may interleave neural-evaluated atoms with ordinary symbolic literals. The symbolic literals act as hard join conditions: they gate which groundings can fire but carry no probability mass or gradient. The head probability therefore factors as a 0/1 indicator of the symbolic condition times the neural probability scaled by a learned per-rule weight $\sigma(w)$, and gradients flow only through the network outputs and the rule weight, never through the ground facts.

Existential joins. When an existential (non-head) body variable is a neural predicate's input, xlog grounds that predicate over the *real* join domain *inside* the circuit instead of stripping the join as a pre-filter: the neural occurrence expands into one circuit leaf per domain event, and provenance OR-aggregates $\exists e.\ \text{neural}(e) \wedge \text{join}(e, \mathit{head})$ at each head binding. This makes "does supporting evidence exist?" rules learnable, reasoning over a whole domain of events instead of only per-instance classification.

Joint multi-rule mixtures. Several same-head trainable clauses combine as a noisy-OR mixture, $P(\mathit{head}_i) =$

$1 - \prod_k \big(1 - r_k[i]\, m_k[i]\, \sigma(w_k)\big)$. Here $r_k[i]$ is a candidate's relational eligibility, $\sigma(w_k)$ its learned rule weight, and $m_k[i]$ an optional graded confidence in $[0, 1]$ carried per binding, for example a fact's confidence trajectory over time. Rule learning thus moves from binary rule eligibility to a continuous, differentiable weighting of competing and cooperating rules.

Learned body gates. Inside that mixture a candidate may also carry a neural conjunct over a per-binding entity feature $\phi(x)$: its eligibility becomes its relational grounding *and* a straight-through-thresholded gate $g_\theta(\phi(x)) \geq \tau$. A gated clause then competes against ungated ones on the same noisy-OR terms, and g_θ trains jointly with the rule weights under the same held-out selector. By default ϕ is detached where it enters the engine: gradient reaches θ and the rule weight but stops at the neural head, leaving the feature producer frozen. Training the producer too is an explicit opt-in per candidate: one flag leaves ϕ attached, so gradient flows on into the backbone that computed it. Only the autograd linkage changes; the values uploaded are identical either way.

The single-clause path uses the verified circuit. Multiple same-head clauses instead select a PyTorch noisy-OR mixture: its per-candidate join probabilities can be compared with the single-clause circuit, but the combined mixture is not certified by that circuit. Engine-mode witness credit is a third route, described next. Section 7 uses that credit route for its neural detector; its relational searches use a CPU harness. Neither constitutes a benchmark of the circuit-based multi-clause path. Section 12 states what that evidence does and does not support.

6.5 Engine-mode credit and candidate selection

Which of the candidate bodies the engine enumerates should the learner keep, and on what evidence? Learning a rule body is not only a gradient problem: it is also a decision about which candidate to keep, and the two need different evidence. Engine-mode training separates them. The engine already enumerates the candidate space (the well-formed pairs of body relations over the program's own binary relations) and already holds the join extensions those candidates read, so training scores that space directly instead of a reconstruction of it.

A candidate's score on a fact is a noisy-OR over the witnesses the engine reports for that fact: a neural body relation contributes a probability per witness, read at the fact's own label, while a purely relational one contributes a fixed $\{0, 1\}$ cover. Candidates combine as a softmax over their logits, and the resulting negative log-likelihood is a single autograd graph, so one backward pass updates the candidate logits and the network behind the probabilities together.

A worked instance. Take one head fact and two candidates for its body, one relational and one neural. The relational candidate covers the fact or does not, so its score there is 1 or 0. The neural candidate's score is the noisy-OR over the witnesses the engine reports for that fact, each read at the fact's own label: two witnesses at probability 0.5 give $1 - 0.5 \cdot 0.5 = 0.75$, above a relational candidate that does not cover the fact and below one that does. Those are training scores, and the arbiter never reads them. It rescores both on the held-out facts by the same witness semantics, drops either whose held-out score falls below the fit gate, and keeps the relational one when the two land within the score quantum; if that narrowing leaves two relational candidates the data cannot separate, it abstains rather than pick one.

The candidate pool is validated against the typed registry of Section 6.1 instead of being trusted by name: a neural relation whose declared arity the credit has no witness semantics for is refused at registration, and a pool that filtering empties reports its per-filter counts instead of silently shrinking. And because raw engine constants index the caller's feature rows directly, that identity is accepted only when the witness domain is exactly the dense row range; a misalignment that would stay in bounds while gathering another event's probability draws a refusal instead of a guess.

Selection on held-out generalization. Training credit cannot distinguish a crisp but coincidental rule from a soft but correct one, even in principle; generalization can. Selection therefore never reads the trained weight. The facts are split into folds; each fold trains on the rest and rescores every candidate on the held-out facts by its own witness semantics; the arbiter ranks the fold-averaged scores. A *fit gate* then drops any candidate whose held-out score falls below a threshold, since a rule that cannot fit held-out data is not a rule.

Ties within the score quantum break toward the relational candidate on Occam grounds, but only when that narrowing leaves exactly one: preferring a relational explanation over a neural one is licensed, choosing among relational duplicates the data cannot separate is not. When neither step is decisive the arbiter returns no rule and says why; abstention is one of the selector's outcomes. The held-out score may be accuracy or F1, and the choice matters: at a rare positive class accuracy sits on the all-negative base-rate plateau, where a candidate that predicts nothing outranks the best real detector, which is why the benchmark protocol of Section 7 selects on held-out F1 instead.

Masked witnesses and withheld evidence. A per-witness mask lets evidence be withheld rather than denied: a masked witness leaves the candidate's index entirely, contributing zero credit and zero gradient instead of a coerced false, and the facts it thereby leaves uncertain are excluded from the held-out score and counted against a coverage gate that runs before the fit gate, so no candidate is killed by facts it was never allowed to see. The channel rides the accuracy axis only and is refused up front beside

the F1 holdout, whose measurability derives from raw labels a mask can hide.

Acceptance without retraining. The same machinery also runs as an acceptance instrument: given an externally trained detector, a frozen entry point scores every enumerated candidate against it with no optimizer, no gradient and no folds and applies exactly the arbiter's gates, so where the cross-validated path asks whether a detector can be trained for a rule, this one asks whether a rule is right given the detector one already has. A training-mode module is refused rather than quietly switched, since batch-norm statistics and dropout would de-determinize the bit-identical scores the entry point exists to provide.

6.6 Term embeddings and differentiable ILP

Term embeddings attach dense, optionally trainable vectors to logical symbols. Where `nn/4` maps perception to discrete facts, embeddings give logical entities a continuous representation that participates in neural computation, with gradients flowing through the embedding lookup: knowledge-graph entity representations, for instance, trained jointly with neural perception.

Differentiable ILP learns first-order rules instead of network weights. Candidate rules are represented as sparse bitmasks over the ground-atom space, credit assignment runs entirely on-device via scatter–gather, and a promotion pipeline decides which rules enter the program.

Four kinds of judgement sit on these two paths, and Table 2 separates them. The promotion entry point requires the host-reporting training mode (`strict_gpu_native=False`); it refuses the default strict mode. Nonconvergence returns before trial compilation. After convergence it compiles and evaluates the trial, runs the six ordinary gates and optional ambiguity scan, and then checks for explicitly supplied held-out examples. With neither held-out positives nor held-out negatives supplied, it returns for manual review; the six gates cannot promote on their own. Ambiguity remains informational, not a promotion veto.

The benchmark runs of Section 7 exercise the last row only. Its permutation-null threshold is not an extra gate but the holdout arbiter's own fit threshold, derived per fold from label permutations instead of fixed at a constant; the promotion gates above it belong to the differentiable-ILP path and are not on that route.

Both paths ride the device-resident bridge of Section 6.3. Where the trainable rule bodies of Section 6.4 learn rule *weights* and per-binding confidences, differentiable ILP learns rule *structure*; the two are complementary. This sparse, GPU-resident formulation avoids the cubic blowup of dense rule materialization. Unlike the trainable-body surface, this path is not exercised on the benchmark of Section 7, and it is a beta subsystem (Section 12).

Table 2: What judges what, on which path: differentiable-ILP promotion or the holdout arbiter of Section 6.5. Nonconvergence stops before trial evaluation; missing explicit held-out examples prevents promotion after the ordinary gates have run.

Judgement	What it judges	Path
Six always-run gates	the rule derives every training positive and no training negative, passes the novel-fact audit, loses no protected fact, meets the configured F1 threshold, and checks typed schemas when enforcement is enabled (off by default)	dILP
Held-out gates	the same rule on each nonempty supplied held-out positive or negative set	dILP
Ambiguity scan	which other candidates also explain the data; reported, never decisive	dILP
Permutation-null fit threshold	whether a candidate generalizes better than shuffled labels do on its own fold	arbiter

7 Learning Event Rules from Perception

The preceding sections build the machinery; this one puts it on an external benchmark. The task is symbolic event-rule induction over video surveillance data, and the perceptual predicate the induced rules depend on is itself learned through the logic credit alone, with no perceptual supervision anywhere in the loop.

The Event Calculus is the standard logical formalism for narrative reasoning over time: a *fluent* is a time-varying property of the world, for instance that two tracked people are `meeting`, and the two event predicates `initiatedAt(F,T)` and `terminatedAt(F,T)` say when something at time `T` makes fluent `F` begin or makes it stop. A domain-independent axiom of inertia closes the loop, so that a fluent *holds at* a time if it was initiated earlier and not terminated since. Reasoning is therefore cheap and generic once the initiation and termination rules are known; the whole difficulty is obtaining them.

Those rules are classically hand-written by a domain expert. The line of systems that learns them instead, inductive logic programming over Event-Calculus theories, learns them from a symbolic input vocabulary that somebody else has already computed. On the CAVIAR video benchmark the learner is handed a relation such as `close(P1,P2,T)`, already reduced from raw tracker coordinates by a hand-chosen distance threshold, alongside per-person activity states. Rule induction thus starts one layer above perception, and the perceptual layer's quality enters as a fixed input that the learning does not explain.

In the runs below the proximity predicate is not given. It is `close_nn`, a small network over the raw pair coordinates, and it is never shown a distance label, a threshold, or any target of its own. Its only gradient arrives through the induced rules: a candidate clause containing `close_nn` in its body is scored by how well the resulting Event-Calculus theory explains the frame-level annotations, and that score is what propagates back to the network's weights.

This is the trainable rule bodies of Section 6.4 on a real benchmark instead of a demonstration: the neural atom sits inside a rule body, and the symbolic literals beside it act as hard join conditions that gate which groundings fire without carrying gradient. These runs take the engine-mode credit path of Section 6.5: the rule learner's own negative log-likelihood is the network's loss, so structure search and weight training are one differentiable object rather than two pipelines in sequence. The compiled route of Sections 6.3 and 6.4, with its machine-checked equivalence certificate and its zero-copy DLPack export, is the one the MNIST-addition measurements of Section 10 exercise; `close_nn` here is trained by a different route.

7.1 Protocol

Two targets are searched throughout this section. The *direct* target asks, per timestep, whether the fluent holds; the *Event-Calculus* target asks when it is initiated and terminated, and infers holding through inertia. They carry different class balances, and the guarantees below are stated per target. Except where a table says otherwise, F1 is micro-averaged over pooled per-fold counts, as the published evaluations compute it.

Each guarantee below was fixed before the run it governs and covers that run alone: the corpus was already in hand while the gates were being designed.

Leak-free scene-family splitting. The distributed CAVIAR corpus used by the published Event-Calculus learners is a dump of 26 video segments, and it is not clean. A frame-level audit of every `meeting` transition event in the dump against the original ground-truth XML classifies 21 of 25 events as real, 3 as duplicates and 1 as a splice artifact. Two source videos appear in both the dump's train and its test file, and one video alone contributes 855 duplicated gold pair-frames. Cross-validating over segments therefore permits same-scene transfer. The clean protocol folds over *scene families* recovered from the source XML instead, so no scene appears on both sides of a split.

Recall-aware holdout. Scoring candidates by held-out accuracy is useless at this class balance (roughly ten positive transitions against tens of thousands of rows), because every candidate sits on the all-false base-rate plateau, and the ranking provably inverts: the best real detector scores below the empty theory. Candidate selection in the Event-Calculus searches uses per-fold held-out F1 instead. The direct per-timestep target does not have this problem, since it carries thousands of positives per fold and not a handful of transition events, so its searches are left at their defaults, held-out accuracy with a fixed 0.75 fit gate. This guard and the next therefore govern the Event-Calculus route only.

Permutation-null fit gate. A candidate must beat what label shuffling alone achieves on its own fold. Each fold derives its own threshold from 1000 label permutations (seed 7) as the 95th percentile of the pool-maximum mean per-fold F1, computed per target and on the training side only. A theory whose clauses do not clear that bar is not returned: the search abstains and reports an empty theory. Abstention is a first-class outcome of this protocol.

Two fold structures. The direct per-timestep runs reuse the three folds the distributed dataset file ships, whose datapoints are fixed-length windows of consecutive frames for one person pair (the *windowed folds*); the Event-Calculus runs fold over whole video segments, or over scene families under the clean protocol.

7.2 Results

Perception learned through logic credit. On the windowed-fold protocol with a direct per-timestep target, the search finds the same two-clause theory on every fold in both modes (Listing 12):

```
holdsAt_meeting(PP,T) :- both_inactive(PP,T), close*(PP,T).
holdsAt_meeting(PP,T) :- both_active(PP,T),   close*(PP,T).
```

Listing 12: The two-clause theory the search returns on every fold, in both the relational and the neural mode.

where `close*` is the precomputed geometric predicate in the relational mode and the learned `close_nn` in the neural mode. Held-out F1 per fold is 0.9215/0.6804/0.7695 with the precomputed predicate (mean 0.7905) and 0.9215/0.7918/0.7695 with the learned one (mean 0.8276). The learned detector matches the precomputed predicate's result exactly on two folds and outperforms it on the third. That is this section's main positive result: on these windowed folds, a perceptual predicate trained through symbolic credit stands in for hand-set geometry without loss, and rests on no published comparison.

Protocol-matched cross-validation. Under the full Event-Calculus target, the combined 26-segment corpus (32,360 co-visible pair-frames, 1,833 gold `meeting` frames) was cross-validated ten-fold by video segment with every gate re-derived inside each fold. The result is precision 0.6580, recall 0.8271, F1 0.7329. The same initiation clause, `both_active & close`, is selected on all ten folds, each time over that fold's own permutation null; the termination theory is empty on all ten, so the fluent persists by inertia to the end of each co-visible pair-run.

Published figures on the same corpus. This protocol matches the published Event-Calculus learners on the axis that matters most for the metric: F1 is computed on `holdsAt` inferred through inertia, on the same distributed corpus, with the same micro-aggregation. The published figures for `meeting` on that corpus are 0.735 for hand-crafted rules, 0.782 and 0.792 for OLED [22] (the latter as reported by its authors, the former as re-run in the WOLED paper [23]) and 0.887 for WOLED-ASP; they are tabulated beside ours in Section 10. Ours sits just under that group's lowest figure. We draw no ordering from any of these differences, in either direction. Five protocol deltas separate the rows and they do not all point the same way: the fold structure, the scope of the learned theory, the body-length limit and the way the reported settings were chosen all differ, and the shared metric is the only reason the comparison is drawn at all. Table 11 prints the five deltas in full beside the figures.

A second vocabulary iteration. Two pair-level transition relations were added to the Event-Calculus vocabulary, one for a pair crossing the proximity threshold outward and one for strictly growing distance, and the identical folds, seeds and gates were re-run. The result rises to precision 0.7357, recall 0.8260, F1 0.7782 (1514 true positives, 544 false positives, 319 false negatives), with a termination theory now learned on seven of the ten folds. That figure falls inside the 0.735–0.887 span the published systems occupy, but it must never be read as the headline: its vocabulary was chosen *after* the first run's test-side failure mode was known, on the same folds. Everything else about it is pre-registered; the vocabulary is not. Against published estimates selected as the best of several tried, the smaller number declared in advance is the stronger scientific claim, which is why it leads here and 0.7782 appears only with this label attached.

The learned detector under the Event-Calculus target. Substituting the learned `close_nn` detector for the precomputed proximity predicate inside the Event-Calculus initiation search does not survive cross-validation: precision 0.1250, recall 0.0005, F1 0.0011 over the same ten folds (Table 12 gives the counts). Only one fold commits a clause, the same `both_active & close_nn` shape the geometric predicate selects there, and probed directly against held-out ground-truth proximity that clause scores precision 1.0 at recall 0.27, so the network has learned the geometric relation. The failure is one of scale in the selection rule, not of perception: the initiation search scores coverage against transition *events*, a handful per fold, and a probability-thresholded gate newly covers fewer of them than a deterministic predicate does, so nine of ten folds reject the candidate for insufficient new coverage.

The leak-free protocol. Re-run on the deduplicated, scene-family-grouped corpus, the `meeting` fluent yields F1 0.0 on both routes, and mostly by committing a rule that earns nothing rather than by declining to commit one. The direct route selects `both_inactive & close` on nine of the ten folds, and that clause returns no true positive anywhere (0 tp, 106 fp, 1,812 fn). The Event-Calculus route commits `both_active & close` on one fold, whose 120 false positives are its entire error mass (0/120/1,812), and abstains on the other nine, recording insufficient new coverage or an abstaining selector; its termination search returns nothing on any fold, most often because no body passed that fold's fit gate. Both routes produce false positives, which an empty theory cannot: the search ran, and what failed is transfer.

The reason is visible in the data. One scene family carries 1,323 of the 1,812 `meeting`-positive frames and, under honest grouping, sits in exactly one fold; the clause that explains those frames holds *only* there, covering zero of the 489 positives on all eight remaining `meeting`-bearing segments. Both outcomes follow from that. Where the clause is committed, it is committed on a training side that contains the concentrated family, and it covers nothing outside it. The one fold on which the direct route abstains is precisely the fold that holds that family out, leaving a training side on which the clause covers 0 of 489 positives and 106 negatives.

Where the gate abstains, it is because the candidate's held-out score has collapsed for the same reason. The statistical bar itself did not rise: the per-fold null thresholds under this protocol are essentially those of the like-for-like distributed-corpus run. Training with the family learns a clause that transfers nowhere; training without it cannot learn the clause at all. With eleven observable `meeting` initiations and five `moving` ones, the clean corpus does not carry enough evidence for a cross-validated theory in either direction.

The machinery is not broken under this protocol. On `moving` the direct route still selects the canonical published rule shape on nine of ten folds (micro F1 0.4450), and the `meeting` clause applied to the concentrated family's own fold, an in-family fit and not a transfer test, scores frame F1 0.890 there. What it establishes is narrower and more specific than an abstention: a two-literal state vocabulary does not carry CAVIAR `meeting` across scene families at all.

7.3 Discussion

Two of the findings above concern the method. A perceptual predicate can be learned end-to-end through symbolic credit alone and stand in for hand-set geometry without loss on a real benchmark. And Event-Calculus rules can be induced under fully pre-registered gates and land within a couple of points of the range published Event-Calculus learners report on the corpus they share.

The other two concern that corpus. It carries a measurable duplication defect, and under a leak-free split of it the induced `meeting` rule does not transfer: the search still commits the canonical clause on most folds, and that clause scores zero out of family.

Run	P	R	F1
Windowed folds, direct target (mean of 3 folds)			
precomputed `close`	—	—	0.7905
learned `close_nn`	—	—	**0.8276**
Distributed corpus, 10-fold CV, EC + inertia			
pre-registered	0.6580	0.8271	**0.7329**
+ termination relations	0.7357	0.8260	0.7782
learned `close_nn`	0.1250	0.0005	0.0011
Clean protocol (scene-family folds)			
`meeting`, EC route	0.0	0.0	0.0
`meeting`, direct route	0.0	0.0	0.0
`moving`, EC route	0.0	0.0	0.0
`moving`, direct route	0.5334	0.3817	0.4450

Table 3: Runs reported in this section. Micro F1 over pooled per-fold counts, except the windowed rows, which are fold means. The *+ termination relations* row is the second, adaptive vocabulary iteration. The clean-protocol zeros are scored predictions, not empty theories, except the `moving` Event-Calculus row, whose initiation theory is empty on every fold; the census behind them is in the text. Published figures for the same benchmark are tabulated separately in Section 10.

None of this ranks the system against the published learners (Section 12). Termination programs are not learned in the pre-registered run either, and the empty termination theory is the dominant source of false positives there. Nor is the learned detector a drop-in replacement for the geometric predicate everywhere: under Event-Calculus cross-validation it is not. And it is one benchmark in one domain. What the section does show is that the structure search and the perception underneath it can be one differentiable object instead of two systems in sequence.

8 Rule Induction at Scale: Maritime Rendezvous

The previous section ends on an evidence problem: under a leak-free split CAVIAR carries eleven observable `meeting` initiations, too few for a cross-validated claim about rule learning. This section moves the same machinery to the other public corpus the Event-Calculus learners report on, the Brest AIS maritime stream [24, 25], whose target fluent `rendezVous` (two vessels close together, stopped or moving slowly, in open water) carries 3,548 gold intervals.

It is large enough to separate three questions CAVIAR could not. Does a *crisp* rule search, selecting clauses that each fire with weight one as in Section 7, land where the declared ceiling says it will? How much is gained by *weighting* the clauses of the same gated pool instead of selecting them? And what does it cost to learn those weights in one chronological pass over the stream? The six runs of Table 5 answer them, covering the published systems' three training settings: enumerated rule search, weighted clauses, single-pass weight training.

Two of the three answers came out better than we expected.

8.1 Corpus and labels

The inputs are two public exports of RTEC [26], a hand-written Event-Calculus rule engine, over the Brest AIS stream: the composite events it recognised (Zenodo record 2557290) and its own input, the *critical points* of the compressed trajectories [25], the timestamps at which a vessel stops, turns, changes speed or loses signal. Those points are the only temporal grid the corpus carries. A deterministic converter reduces the two into a *pair-time corpus*: one row per (vessel pair, timestamp), eleven relations that hold or do not for that pair then, and the gold label. Table 4 sizes it. The conversion uses no randomness: the corpus is a pure function of the archives.

Table 4: The pair-time corpus, as the converter leaves it. The negative pairs are drawn by fixed stride from the 2,014 pairs that come within proximity but carry no interval.

Quantity	Value
Rows (vessel pair × timestamp)	454,858
Vessel pairs	806
carrying a `rendezVous` interval	302
negatives, by fixed stride	504
Gold intervals	3,548
Positive rows	3,579
as a share of all rows	0.79%

Three properties bound what a number measured on it can mean. The supervision is synthetic: the gold labels are a hand-crafted RTEC rule's output over the same interval streams the vocabulary comes from, so their perfect alignment with those streams says the conversion is faithful, not that the labels are right. The published learners share this regime; the WOLED paper notes that the maritime ground truth is synthetic and its learning curves consequently alike [23]. What every system here measures is the *reconstruction of a known rule* under a vocabulary that does not contain all of it.

The vocabulary is incomplete by construction. Its eleven pair-time relations (`proximity`, `both_lowspeed`, `both_stopped_far`, `both_open_sea`, `became_proximate`, and so on) omit the gold rule's two remaining discriminators, its 240 s minimum-duration threshold and its exclusion of tug and pilot pairs, so perfect reconstruction in the base vocabulary is impossible.

And the grid is sparse: ≈1.009 positive rows per gold interval, the covering row almost always the interval's start boundary, so pointwise and interval F1 coincide here (Section 8.4).

8.2 Protocol

Metric. The primary metric is pointwise F1 on the positive class, F1 over individual rows, never accuracy, which the empty predictor saturates here. Interval figures accompany it, a gold interval counting as matched when a predicted row falls inside it in the same episode of the pair's timeline, and symmetrically.

The declared ceiling. Each run's hypotheses and parameters were fixed before it was executed, beginning with the ceiling the base vocabulary imposes. The gold rule's definitional body is `proximity & both_low_or_stopped & both_open_sea`, the middle literal being the disjunction of `both_lowspeed` and `both_stopped_far`; all three are in the vocabulary, so the body is expressible either way. It reaches recall 1.0 at pointwise precision ≈0.49, hence F1 ≈0.66: the two inexpressible discriminators are exactly what separate its false positives from its true ones.

A direct evaluation of that body on the archives, the *ceiling probe*, hits the operating point exactly: F1 0.6599, no false negative (Table 6). It also decomposes them: 733 in negative pairs, the pair-exclusion class, 2,956 in predicted runs shorter than 240 s, and *none* outside the two pre-identified discriminators.

Folds are vessel pairs. A pair's rows are never split across folds, because the positive mass is concentrated: the top pair carries 33.4% of all positive rows and the top four 60.2%, so a finer split would leak the dominant encounter across the boundary. The protocol uses five folds where CAVIAR uses ten, assigned by a deterministic greedy longest-processing-time rule with no seed, stratified by positive mass, none differing from another by more than the largest pair's count. The consequence is stated openly: held-out fold 0 is the top pair plus one negative pair, so per-fold figures are necessarily unequal and the micro number never stands alone.

Search and gate as on CAVIAR. The crisp run uses the relational sequential-covering search of Section 7 under the CAVIAR limits: bodies of at most three literals, at most four clauses, per-fold held-out F1 for candidate selection, at least two newly covered positives per clause. Its permutation-null fit gate is the CAVIAR one: 1,000 label permutations per fold, threshold at the 95th percentile of the pool-maximum per-fold F1, derived on the training side. One seed, 7, drives the permutations and the candidate-selection holdout. The target is the direct per-row positive label; induction through inertia is out of scope here, with almost no interior rows to fill.

Weights. The weighted runs keep vocabulary, folds and gate, replacing crisp selection with weights. The body pool is every conjunction of one to three literals passing the same per-fold permutation-null procedure, thresholds re-derived per run, each body carrying one weight. A row's score is the noisy-OR $1 - \prod_c \big(1 - \sigma(w_c)\,\mathrm{cover}_c\big)$ of Section 6.4, every clause purely relational, trained by binary cross-entropy with Adam and thresholded at 0.5: 300 steps at learning rate 0.05 from every weight at -2.0, so every clause starts switched off ($\sigma(-2) \approx 0.12$). The single-pass variants replace that optimisation with *one* pass over the fold in ascending global time, its pairs interleaved as in a stream, in mini-batches of 1,000 rows, one Adam step per batch, at the same rate. Adam is kept instead of WOLED's AdaGrad, so exactly one variable separates the regimes. Pool and gate are still computed on the training side, which makes this single-pass *weight* learning, semi-online in that only the weights see the stream; the structure is never learned online.

8.3 Results

Crisp search on the base vocabulary. The first run is the CAVIAR search unchanged. It lands where the ceiling mechanism predicted, precision near one half at recall near one, for micro pointwise F1 **0.6746** (Table 5).

On every fold it commits both disjunctive branches of the gold body: `both_open_sea & both_stopped_far & proximity` and `both_lowspeed & both_open_sea & proximity`. Three folds add a further clause, on two of them the subsuming disjunction relation itself. No clause is ever accepted below its fold's null gate.

The learned operator is not literally the definitional body, its precision being 0.5109 against the body's 0.4924, so the evidence is about the mechanism, not operator identity. The per-fold spread of Table 6 is the pair-concentration effect: the fold with the top pair's long clean encounters scores highest, while folds of many small pairs concentrate the duration-threshold false positives. Improvement past this run has to come from expressiveness rather than tuning.

Weighted clauses on the same pool. The second run changes one thing: the pool's clauses are weighted rather than selected. We expected it to stay within $[0.66, 0.70]$, the ceiling being a property of the vocabulary.

The hypothesis was rejected in the favourable direction. Micro F1 is **0.7398**, +0.0652 over the crisp run on the same vocabulary, folds and gate, at markedly higher precision and lower recall; the abstract and Section 1 round that gain to 0.065.

The expectation was wrong for an instructive reason: the 0.66 ceiling is the F1 of the definitional body's operating point, crisp reconstruction at recall near one, and weights are not confined to it. Training moved the operating point to high precision at lower recall, which carries more F1.

Fold by fold the picture is mixed. The weighted run wins the micro figure and the median but only two of

the five folds, and its weakest fold is worse than the crisp run's. That weights over the same clauses beat crisp selection of them is the gain the published line attributes to weighting [23].

Both runs again with the duration relation. The third and fourth runs add one relation, `sustained_240`: a row carries it when `proximity`∩`both_low_or_stopped`∩ `both_open_sea` holds continuously through a stretch of at least 240 s containing it. The constant is the gold generator's own, so the addition is language parity with RTEC, not a look at the data.

The ceiling probe was re-run on the widened vocabulary before any cross-validated run, against an expectation of ≈0.907 on the assumption that duration removes only the short-run false positives. The ceiling came out higher, at F1 0.9969, again with no false negative.

Duration also eliminates at least 711 of the 733 negative-pair false positives, the excluded pairs' body-condition episodes being almost always shorter than 240 s. The bound is a lower one because the 22 false positives the widened probe still makes are not decomposed by class. Two consequences were predicted from that: the pair exclusion is largely redundant with duration here, and the duration vocabulary saturates, so the informative weighted-against-crisp comparison is the base-vocabulary one.

Both held. Crisp search on the widened vocabulary lands exactly on the derived ceiling and the weighted run equals it on every fold, so the directional hypothesis, weighted no worse than crisp on the median, holds trivially, with no headroom for weights.

Weighted clauses learned in a single pass. The last two runs change only how the second run's weights are trained, replacing batch optimisation with a single chronological pass. They use the base vocabulary only: `sustained_240` at time t depends on how long the current interval will last, a leak in a streaming setting, and was excluded before the run.

Three hypotheses were fixed beforehand: the single pass should lose at most 0.04 micro F1 against the batch run, with anything below 0.70 a negative result; a *reverse* chronological pass should land within 0.02 of the forward one; and the prequential error, scored on each batch before that batch is trained on, should fall along the stream.

The measured degradation is zero. The single pass reproduces the batch run's thresholded predictions exactly: the same counts on every fold and, as re-scoring with the stored weights confirms, the same decision on every row, at the same micro F1. The reverse pass is identical as well, and the prequential error falls from the first half of the stream to the second on all five folds in both orders (forward, fold 0: 0.0078 → 0.0036). Halves are the one criterion of these runs settled as the results were written up instead of before them.

Identical predictions are a statement about the decision structure, not evidence that the single-pass path silently fell through to the batch one: the weights themselves differ substantially between the three regimes (Table 5). The mechanism is simpler than robustness to the training regime. In all fifteen fold-by-regime cases exactly one body has $\sigma(w) > 0.5$, always `became_proximate & both_open_sea & both_stopped_far`, and the second-ranked body stays between 0.15 and 0.46, so the noisy-OR of the others never reaches the threshold on any held-out row. All three trainings reduce to the same crisp rule, and the predictions agree because that rule does.

Run	P	R	F1	F1, fold median
Base vocabulary (11 relations; declared ceiling ≈0.66)				
crisp search	0.5109	0.9925	0.6746	0.6596
weighted, batch	0.8715	0.6426	**0.7398**	0.6928
weighted, forward pass	0.8715	0.6426	0.7398	0.6928
weighted, reverse pass	0.8715	0.6426	0.7398	0.6928
Duration vocabulary (+`sustained_240`; ceiling 0.9969)				
crisp search	0.9939	1.0	0.9969	0.9950
weighted, batch	0.9939	1.0	0.9969	0.9950

Table 5: Maritime `rendezVous`: five-fold cross-validation over whole vessel pairs, seed 7. P, R and F1 are pointwise micro-averages over pooled per-fold counts; the last column is the median of the five per-fold values. The three base-vocabulary weighted rows are identical row by row, though their weights are not: fold 0's top body carries $\sigma(w) = 0.6476$ in batch, 0.5880 in the forward pass and 0.8398 in the reverse, and the L_∞ distance between the logit vectors of any two regimes is 1.06–2.95. Interval-level F1: 0.6772 (crisp, base; 3,545 of the 3,548 gold intervals matched), 0.7435 (weighted, base), 0.9970 (both duration rows). No published figure appears here (Section 8.4).

The agreement is fragile. The highest score of a row *not* covered by the top body reaches 0.493 in fold 1 under the batch weights, 0.007 below the threshold.

That fold has 1,270 rows between 0.3 and 0.5: a small change of learning rate, step count or initialisation could push the second body over and move the counts by hundreds of rows. The declared scope is narrow. Prediction identity holds for this pool, corpus and threshold, hangs on the discrete structure of a thresholded decision, and is not a theorem.

8.4 Discussion

The published figure for `rendezVous` on this corpus is 0.98 for OLED and both variants of WOLED, MLN- and ASP-based alike, with theories of eighteen literals, measured on a train/test split of halves [23]. A second published experiment, on a six-sequence fragment, compares WOLED-ASP with batch learners and reports figures from 0.91 to 0.97 in a bar chart. None of them

Base vocabulary	**f0**	**f1**	**f2**	**f3**	**f4**
crisp search	0.9933	0.4690	0.6685	0.5699	0.6596
weighted, all three	0.9346	0.4237	0.8000	0.6928	0.5342
Pooled counts	tp / fp / fn				
ceiling probe, base	3,579 / 3,689 / 0				
crisp search	3,552 / 3,400 / 27				
weighted, all three	2,300 / 339 / 1,279				
ceiling probe, duration	3,579 / 22 / 0				

Table 6: Per-fold F1 on the base vocabulary and the pooled counts behind Table 5. The three weighted regimes share a row because their predictions are identical. The probe rows evaluate the gold rule's definitional body directly; the duration probe's 22 residual false positives are what its 0.9969 ceiling reflects.

stands beside the rows of Table 5, because the two sets of numbers are not on one axis.

Theirs live on a dense grid, the critical points of every vessel with inertia filling each interval's interior. Ours live on the sparse per-pair grid the converter produces, and we do not reproduce the published per-timepoint reading on it. A split of halves is not a pair-level five-fold cross-validation either, and for the same reason we do not compare pass times with published training times.

What is comparable is comparable in kind only. The published systems saturate on their preprocessing of the same stream, and ours too once the duration discriminator is expressible, at the declared ceiling. And the gain the published line reports between OLED and WOLED on CAVIAR, and attributes to weighting, is reproduced here as +0.0652 micro F1 with everything else held fixed, on a corpus with 3,548 gold intervals instead of eleven events, and again without loss when the weights are learned in one pass.

Within those bounds the findings are concrete. The crisp search reconstructs the gold body on every fold of a corpus two orders of magnitude richer in positive events, and lands on the declared ceiling. Weighting the same pool adds 0.0652 micro F1 and 0.0332 to the median against the expectation declared for it, on two folds of five. Duration saturates both search modes and makes the pair exclusion largely redundant. And single-pass weights reproduce the batch predictions exactly, for the reason named above.

No ranking against OLED or WOLED on `rendezVous` follows from any of it (Section 12). Nor does it say anything about detecting rendezvous in the wild: every number here is a rule-reconstruction figure. It does not demonstrate online structure learning, the pool and gate being computed on the training side, nor a general robustness of thresholded noisy-OR predictors to the training regime. And it exercises none of the GPU substrate: every run here is a deterministic CPU procedure.

What it does show is that the induction protocol of Section 7, its declared gates, pair-level folds and permutation nulls, carries intact to a corpus where its verdicts are measurements and not anecdotes. There, weighted clauses and single-pass training, the two mechanisms the published line rests on, are measured in isolation and behave as their authors report.

9 Epistemic and Other Reasoning Modes

The substrate generalizes beyond what is *true* or *probable*. This section covers epistemic reasoning over world views, probabilistic aggregates, and well-founded semantics. These routes reuse CUDA execution services but do not share one residency contract: canonical probabilistic provenance construction, including its well-founded analysis, runs on the host before device compilation.

9.1 Modal surface and the epistemic IR

Epistemic logic programming asks what is *known* or *possible* across the multiple stable models a program admits—the natural setting for introspection and reasoning under incomplete knowledge. A program may use four modal literals over a predicate: `know`, `possible`, `not know`, `not possible`. Instead of rewriting them away at parse time, the compiler preserves them in an Epistemic IR (EIR) between the AST and execution planning. The high-level dispatcher classifies modal dependencies before selecting Generate–Propagate–Test, a mode-specific positive-cycle fixpoint, or GPU-backed WFS. The accepted surface spans finite nested modal chains, epistemic integrity constraints (which prune candidate world views on the GPU), rules that couple several epistemic predicates, same-name multi-arity modal predicates, and recursive epistemic execution.

9.2 World-view semantics

A world view is a non-empty set of stable models. `know p` holds when `p` appears in every model of the world view; `possible p` holds when it appears in at least one. Two semantics are selectable. Under the default FAEEL (Founded Autoepistemic Equilibrium Logic), a circular derivation contributes a tuple only when independent founded support exists. A program containing only `p :- possible p.` therefore executes to an empty founded extension instead of failing with an unsupported-construct diagnostic. A Gelfond-1991-style compatibility mode admits that self-supporting tuple. Non-epistemic programs evaluate identically under both.

9.3 Epistemic execution routes

After EIR construction, acyclic modal programs follow a Generate–Propagate–Test schedule whose phases run

as CUDA kernels. *Generate* enumerates bounded candidate assumptions as device bitsets; *propagate* prunes candidates that contradict known or rejected facts; *test* validates survivors against the program's world views, checking stable-model tuple membership on-device per arity. Validating a candidate on this route reuses the on-GPU CDCL solver already built for compilation verification (Section 5), the same substrate that serves satisfiability, bounded MaxSAT and assumption-based queries, and accepted world-view evidence feeds the existing GPU-native exact path.

A positive FAEEL dependency cycle reduces to ordinary rules and reaches the founded least fixpoint in the existing recursive engine. A supported Gelfond-1991 cycle, whose selected positive `possible` gates must use their rule heads' own terms and stay within one recursive dependency component, instead descends to the greatest compatible set: the runtime relaxes those gates for an upper bound and then reevaluates until GPU set comparison over the intensional relations reports no further change (Section 12). Supported cycles through negation take the GPU-backed WFS alternating fixpoint instead, and recursive negation or aggregation inside a compatibility component is rejected, because either would make the descending operator nonmonotone.

The reduced ordinary work reuses the optimizer, WCOJ planner and helper-splitting machinery of Section 4, and *epistemic splitting* decomposes eligible acyclic programs into independent components solved separately and recombined. No CPU fallback is provided for the accepted data-plane hot paths: a construct these routes do not support is rejected with a typed error; no host implementation serves it. Transfer budgets track data movement; the host sequences Gelfond-1991 refinement and WFS iterations, while relation evaluation, snapshot copies and convergence comparisons remain GPU-backed.

9.4 Probabilistic aggregates

Finite aggregate queries (`count`, `sum`, `min`, `max`, `logsumexp`) are supported by exact provenance/PIR inference, subject to exact-domain caps that reject domains too large for tractable enumeration. Those caps are not Monte Carlo limits. The resident Monte Carlo planner rejects aggregates; selecting that engine is not a general resident alternative for an exact aggregate that exceeds its cap. For small finite `count` domains, *aggregate lifting* replaces naive world enumeration with exact cardinality dynamic programming over a compact state set (collapsing, for example, a 2^{17}-outcome enumeration into a few hundred lifted states with identical results) and accepted `count` aggregates dispatch a GPU-native exact evaluator rather than a CPU finite-world shortcut, computing each supported count probability on the GPU after host setup and then reading its result scalar. This fast path is not a ZERO TRANSFER operation.

9.5 Well-founded semantics

Programs that violate stratification, such as the classic `p :- not q. q :- not p.`, do not have a unique two-valued least-fixpoint interpretation. For supported shapes, xlog instead assigns three-valued truth (true, false, undefined) through well-founded semantics, computing an alternating fixed point: mark the greatest unfounded set false, derive consequences true, and repeat until stable. For probabilistic programs, true atoms receive normal probability and gradient while false and undefined atoms are assigned probability zero, matching the conservative treatment of non-stratified programs. The two callers have distinct implementations. Canonical probabilistic provenance analysis grounds supported rules and calls a host-side alternating-fixpoint evaluator before device compilation. Epistemic dependency dispatch instead selects its supported GPU-backed WFS plan; other cyclic shapes return a typed diagnostic. A probabilistic program using host WFS is not a zero-transfer epistemic GPU run.

10 Evaluation

This section reports measurements for xlog's deterministic, probabilistic, and neural-symbolic subsystems. The subsystem measurements in Sections 10.2, 10.3 and 10.7 are *single-system ablations of xlog against its own baselines*: the first two on development hardware, the runtime-optimization pair on a cloud GPU that the artifact names. Section 10.4 then adds controlled head-to-head comparisons against external engines (Scallop, ProbLog2, Soufflé), Section 10.5 tabulates the Event-Calculus rule-induction runs of Section 7 beside the figures published for the same benchmark and states why the maritime runs of Section 8 are not tabulated beside theirs, and Section 10.6 isolates xlog's two design costs, verification and residency, from ordinary GPU acceleration. Section 10.9 describes three further runtime capabilities for which no timing is claimed.

10.1 Methodology

The subsystem ablations were collected on an NVIDIA RTX PRO 3000 (Blackwell, 12 GB VRAM, compute capability 12.0). The head-to-head comparisons of Section 10.4, the overhead isolations of Section 10.6 and the runtime-optimization ablations of Section 10.7 ran on cloud GPUs, as stated with each, and the neural Event-Calculus runs of Section 10.5 on an NVIDIA A40. Each cloud artifact records its own device, driver and CPU quota. The seven head-to-head and overhead artifacts use one recorded engine build. Their differences from earlier records do not isolate hardware alone: the reconstructed runs also choose previously unrecorded generator parameters and seed values, so neither fixture identity nor a

hardware-only comparison is established. For the two runtime-optimization fixtures, earlier records date from the v0.8.6 campaign, so hardware and engine version both changed. The relational searches of Section 10.5 and Section 8 are deterministic CPU procedures; the maritime weight training runs on CPU as well. Rust crates are built in release mode; CUDA modules are staged cubin-first with PTX fallback and explicit JIT warm-up; Python components use `pyxlog` with CUDA-enabled PyTorch. Native query-kernel targets use Criterion.rs; Python training and comparison scripts use their own runners. Table 7 separates their protocol scopes; warm-up and seeding must be read from each recorded run.

Per-run measurement data is distributed with the paper for the head-to-head comparisons and overhead isolations, the circuit-cache ablation, the runtime-optimization ablations of Section 10.7, and every rule-induction run of Sections 7 and 8. Each other figure below is stated together with the fixture, protocol and aggregation that produced it.

These are measurements of the revisions and environments recorded in the artifacts. They are not a fresh qualification of later source changes. In particular, the CAVIAR neural runs use engine-enumerated witness credit, its relational runs and all maritime runs use CPU procedures, and none establishes a current complete native learning, restoration, or ZERO TRANSFER result.

Table 7: Protocol scope; settings belong to each benchmark, not one universal harness.

Setting	Value
Native WCOJ Criterion target	explicitly selects 10 samples; other Criterion settings are not overridden there
Retained WCOJ ablation	20 warm-up windows and 10 timing windows, as specified in Table 8
Python comparisons	runner-specific warm-up, repetitions, and seed lists recorded with each artifact
Synthetic triangle fixture	deterministic LCG key generation; not a universal data or training seed protocol
Acceptance	no universal three-iteration warm-up, 0.10 significance, or 5% noise gate is established

10.2 WCOJ vs. the binary-join plan

This ablation asks how much the worst-case-optimal join path of Section 4 saves over xlog's own binary-join plan on the skewed cyclic query the subsystem exists for. The query is a single three-way triangle join, the classic worst-case-optimal pathology; Table 8 gives the fixture and the timing protocol.

The keys are generated by four fixture variants named for the workloads they stand in for (call-graph edges, Andersen points-to [27], `ddisasm`-style disassembly [28], and a neural-symbolic mining analog) whose size parameters coincide at this scale, so the four cells are repeats of one workload and not four distinct ones. What the fixture does establish is skew: every join variable has a small hot domain, so the binary-join plan materializes an intermediate far larger than the final result, which is the regime WCOJ exists to address. WCOJ speedups depend strongly on input size and skew, and synthetic fixtures systematically overstate them, this one included, so the ratio below is the gain on the shape the subsystem targets, at this scale.

Table 8: Triangle-join fixture and timing protocol for the WCOJ ablation.

Item	Value
Query	three-way triangle join, one multiway join
Key generation	four fixture variants, coinciding size parameters
Input	three relations of 65,536 rows each
Output	≈4.19M triples
Warm-up	20 windows
Reported time	mean of 10 timing windows
Inner iterations	20 per window (WCOJ), 40 per window (binary join)
Dispersion	coefficient of variation reported, not gated on
Correctness gate	identical row sets on both paths before any timing
Peak allocation	about 2.3 GB, within the device's 12 GB budget

Figure 4 reports the speedups: a 27.96× geometric mean, with the four cells spanning 26.6× to 29.6×. Since the variants coincide at this scale, that spread measures run-to-run variation and not robustness across distinct workloads. Section 10.4 reports the external counterpart, a fused-counting comparison against Soufflé at millions of tuples.

In the retained benchmark protocol, uniform or empty cases select the binary-join chain. Current runtime defaults use cardinality costing; the skew classifier is a separate selectable model.

10.3 Circuit caching and training

This ablation asks how much of neural-symbolic training time circuit caching (Section 5) removes by amortizing D4 compilation across epochs. Training is measured on MNIST single-digit addition; Table 9 carries both the full-run timing, in which the first epoch pays cold-start compilation and verification while warm epochs run



Figure 4: WCOJ vs. binary-join speedup on the four key-generation variants of the triangle fixture (Table 8). Single-system ablation; the dashed line is the geometric mean.

only the cached forward/backward kernels, and the cache ablation beside it.

The ablation is a separate experiment from the full run: its uncached arm recompiles the circuit every epoch while its cached arm compiles once, so it isolates compilation amortization, and the apparent speedup therefore grows with epoch count toward the cold/warm ratio instead of being a fixed property. The 512-image protocol isolates cold/warm compilation timing and is deliberately under-trained (near-chance accuracy); held-out accuracy is measured at scale in the Scallop head-to-head (Section 10.4), where the same pipeline reaches ≈95%.

Table 9: MNIST-addition training summary. The timing rows come from a 512-image, 5-epoch training run at batch size 64; the ablation rows come from a separate experiment (512 images, 3 epochs, 3 seeds), whose speedup is the mean of its three per-seed ratios, not the ratio of its means.

Metric	Value	Notes
First epoch (cold)	71.7 s	D4 compile + CDCL verify
Steady-state epoch (warm)	0.25 s	cached forward/backward path
Total training (5 epochs)	72.6 s	
Per-query cost (steady state)	0.97 ms	forward + backward; steady epoch over 256 pairs, cold epoch excluded
Ablation, uncached arm	242.9 s	mean training time, recompiling every epoch
Ablation, cached arm	88.9 s	mean training time, compiling once
Circuit-cache speedup	2.74×	95% CI [2.29, 3.18]

10.4 Head-to-head comparisons

Table 10 reports controlled comparisons against external engines, one per reasoning mode, on cloud GPUs (A100 80GB and L40S), with matched programs, data and metrics. Each artifact records the pod's CPU quota alongside its GPU, because that quota is what the CPU baselines are given: 26.35 cores for Soufflé here, against 7.65 in the previous measurement of the same comparison. xlog wins where its GPU-resident, bounded-memory design is the point, and does not where a mature CPU compiler is already fast enough on small inputs.

Neural-symbolic: MNIST addition vs. Scallop. With identical MNISTNet, data, metric and seeds, xlog and Scallop reach comparable held-out accuracy: 0.9561 ± 0.0008 against 0.9468 ± 0.0104 on the 20k protocol, and both near chance on the under-trained 512-image protocol. The two systems reach that accuracy by different routes: Scallop scores each fact through a bounded set of proof clauses, while xlog evaluates the exact weighted model count of the compiled circuit on every step. *We make no per-epoch speed claim on this comparison.* Scallop's steady epoch is not monotone in the pod's CPU quota — 31.79 s at 13.6 cores and 54.99 s at 27.2 on the same card, the same wheel and the same seeds — so an xlog/Scallop epoch ratio measured on any one host reports that host as much as either engine. Establishing the ratio would take that curve measured across quotas; the accuracies above stand independently of it.

Probabilistic: exact inference vs. ProbLog2. On five programs (a conditioned Bayesian fragment and probabilistic reach chains) xlog's exact query probabilities match the analytic answers to zero error, identical to ProbLog2 (correctness gate $|\Delta| < 10^{-4}$). On timing, however, xlog is *not* competitive on these small programs. Its per-query time is flat, a fixed GPU launch plus a per-call D4 compile and CDCL verify, while ProbLog2's mature CPU compiler runs each query 45–100× faster. We therefore make no exact-inference *speed* claim; the probabilistic contribution is correctness-equivalent inference with a CUDA-resident circuit data plane and certification of the final smoothed circuit. The host orchestrates the route and observes bounded solver control state, and its fixed costs amortize only at scale or under repeated evaluation.

Deterministic: WCOJ triangle counting vs. Soufflé. On hub-skewed graphs, xlog's fused worst-case-optimal triangle counting, xlog's enumerate-then-count diagnostic, and Soufflé produce the same per-root triangle totals at every size. The comparison uses the median of three full-process executions after one-time native builds: a standalone Soufflé executable is generated once per case with twenty-six jobs, matching the pod's CPU quota, and its 13.05–13.56 s compilation is recorded separately. *The defensible claim is how the ratio moves with input size.* Relative to fused xlog execution, compiled Soufflé is 0.88×, 1.96×, 2.59×, 3.88× and 5.54× across 150k to 1.2M edges: on the smallest graph Soufflé is the faster of

Table 10: Head-to-head comparisons: one external engine per reasoning mode, matched programs, data, metrics and seeds, on cloud GPU pods. The correctness and all-arm completeness gates pass for every comparison.

Mode / task	**Baseline**	**xlog**	**Baseline**	**Ratio**
Neural: MNIST addition; 20 000 training images, 5 epochs, 3 seeds, held-out on the 10k test set	Scallop	acc 0.9561 ± 0.0008	acc 0.9468 ± 0.0104	accuracy parity; no timing claim
Probabilistic: exact inference on five small programs	ProbLog2	0 error; per query flat at 0.79–0.87 s	0 error; per query 0.0085–0.0186 s	45–100× *slower*
Deterministic: triangle counting on hub-skewed graphs of 3.65M to 17.84M triangles	Soufflé (standalone compiled mode, 26 jobs)	counts match at every size; 0.57–0.68 s, nearly flat in the input; peak device allocations 85–1,033 MB	0.51–3.77 s, growing with the input; one-time per-case compilation recorded separately	Soufflé/xlog: 0.88× → **5.54**×

the two, and xlog only overtakes it from 300k edges on. What holds monotonically is that the ratio grows with the input, because xlog's execution time is nearly flat (0.57–0.68 s) while Soufflé's grows linearly (0.51–3.77 s). That flatness needs a caveat the artifact supplies: the engine's own counting time is 25–43 ms of it, so between 93 and 96% of the xlog wall time here is process start, CUDA context creation and Arrow input. What is flat at these sizes is the fixed cost, and the engine time is small enough that this comparison does not resolve how it scales. Fused counting never materializes the triangles and its peak device allocations are 85 to 1,033 MB, while enumerate-then-count peaks at 3,287 to 44,979 MB. All arms complete, both cross-system count checks pass, and the all-arm comparison gate is therefore true. This supports bounded intermediates on these skewed, large-intermediate workloads, not a universal Datalog speed claim: native compilation time is excluded from the execution medians, the measurements are hardware-specific and quota-specific, and on moderate skew the advantage over xlog's own binary join is 2.00–4.95× — measured against a binary arm that completes at every size there, where the earlier artifact had recorded its two largest cases as skipped for blow-up.

10.5 Event-Calculus rule induction

This subsection asks how the induced theories score against the figures other authors have published for the same benchmark, and what the runs that do not favour the system look like. Section 7 states the induction protocol and analyses its runs, and Table 3 there lists them; Table 12 collects their frame-level scores.

Published figures for the same benchmark. Table 11 places our pre-registered ten-fold row beside the four published frame-level `holdsAt` F1 figures for `meeting` on the whole distributed CAVIAR corpus. Our figure is *below the floor of the published span*, by 0.0021: a fully pre-registered run lands just under the lowest published figure, which is the hand-crafted rule set. The second-iteration run reaches 0.7782, which is inside the span, but it is one disclosed adaptive iteration (Section 7); it is not the row in this table.

The direct-protocol reference on the same folds. The run that produced that row also scores a direct per-timestep reference theory on the same ten folds, under the direct protocol's own pre-registered defaults: held-out accuracy, a fixed 0.75 fit gate, the default tie floor. It lands far below the Event-Calculus row beside it (Table 12). The same reference theory comes out of all three ten-fold runs (the pre-registered one, the second iteration and the neural negative), so the second iteration's added vocabulary never reaches the direct search. Its recall is dominated by one fold on which the direct search abstains outright, leaving 855 gold frames undetected. The figure is what those defaults produce on this corpus; it is no ceiling for the direct protocol, whose studied settings produce the windowed-fold results of Section 7.

The remaining runs of Section 7, including the two that do not favour the system, are scored in Table 12 and diagnosed there. One of those rows needs a disclosure that section does not carry.

The `moving` threshold, disclosed. The two fluents carry different canonical proximity thresholds in the published CAVIAR rule set (`meeting` over `close_25`, `moving` over `close_34`), and the `moving` figure above is measured under the canonical `close_34`. An earlier run over the same folds measured `moving` under the `meeting` threshold of 25 and scored *higher* (Table 12). The canonical threshold remains the right headline, and the higher number is not evidence against it: where the clause is selected, widening `close` from 25 to 34 behaves exactly as threshold semantics predict, recall up and precision down.

The micro figure falls for a selection-level reason instead. On the fold carrying 1,546 of the 3,136 gold `moving` frames, the wider candidate pool puts the top two candidates inside the selection tie band, at a margin of 0.0093 against the 0.01 tie floor, and the search abstains there where the threshold-25 run had committed. That figure therefore leaned on a fold whose selection was tie-fragile, which is a reason to report the lower canonical number and the disagreement together rather than either alone.

The maritime figures are not tabulated beside the published ones. The maritime runs of Section 8 are not placed beside the published figure for that fluent; Section 8.4 says why.

10.6 Verification cost and residency benefit

These two isolations ask what xlog's design costs, separated from ordinary GPU acceleration: what the equivalence proof buys with cold-compile time, and what device residency saves in transfers. Both run on an A100 80GB PCIe pod with a 26.35-core CPU quota, and Table 13 carries their measurements.

Verification cost. The cold epoch's compilation time is dominated by the on-GPU CDCL equivalence check rather than the D4 compile. Split on probabilistic reachability chains, CDCL verification accounts for 98.8–99.4% of the cold-compile time and grows with circuit size, while D4 compilation stays between 1.2 and 11.7 ms and is non-zero at every point. The "expensive" cold epoch is thus almost entirely the price of the machine-checked correctness certificate (Section 5), a cost the CPU-symbolic and GPU-Datalog systems surveyed in Section 11 do not pay, because they do not verify.

Residency benefit. To isolate the transfer-free advantage from ordinary GPU acceleration we run the same neural–symbolic pipeline with and without a forced device→host→device round-trip at the neural–symbolic

Table 11: Frame-level `holdsAt` F1 for the `meeting` fluent on the distributed CAVIAR corpus, micro-averaged over pooled counts. Rows are ordered alphabetically by system name, not by F1: the five protocol deltas printed beneath the table do not all act in the same direction. Dashes are figures the cited table does not report.

System	P	R	F1	Source
Hand-crafted rules	0.644	0.855	0.735	OLED [22] Tab. 1(b), `EC_crisp`; the same value appears in WOLED [23] Tab. 2
OLED	0.678	0.953	0.792	OLED [22] Tab. 1(b), whole dataset
OLED, as re-run by other authors	—	—	0.782	WOLED [23] Tab. 2
This work, EC + inertia, pre-registered	0.6580	0.8271	0.7329	10-fold CV by video segment, all gates re-derived per fold (Section 7)
WOLED-ASP	—	—	0.887	WOLED [23] Tab. 2

Protocol deltas, which must travel with this table. (i) The published runs cross-validate over their own windowed interpretations with subsampled negatives; ours folds over whole video segments. (ii) They learn full initiation *and* termination programs; our row commits a two-literal initiation clause and an empty termination theory. Out of its 2,304 predicted-positive frames, 781 of the 788 false positives fall on the folds with detected interior terminations; the other 7 are initiation-side errors persisted by inertia on a fold whose test side detects none. (iii) Their rule language admits longer bodies—bottom clauses averaging some fifteen literals against our three-literal cap. (iv) Their settings are reported as the best among several tried; ours were pre-registered. No number of ours is adjusted on that basis. (v) All rows compute F1 on `holdsAt` inferred through inertia, on the same corpus, the one axis on which the protocols do match, and the reason the rows are placed side by side at all.

Table 12: Frame-level `holdsAt` scores for every rule-induction run of Section 7, micro F1 over pooled per-fold counts. The distributed corpus is the dump shipped with the published Event-Calculus learners and carries the duplication defect measured in Section 7; the clean corpus is its deduplicated, scene-family-grouped form. Dashes are quantities the run does not report separately.

Run	Corpus	P	R	F1	tp / fp / fn
`meeting`, Event-Calculus route, pre-registered	distributed	0.6580	0.8271	0.7329	1516 / 788 / 317
`meeting`, direct-protocol reference, same folds	distributed	0.6864	0.1266	0.2137	232 / 106 / 1,601
`meeting`, Event-Calculus route with learned `close_nn`	distributed	0.1250	0.0005	0.0011	1 / 7 / 1,832
`meeting`, Event-Calculus route	clean	—	—	0.0	0 / 120 / 1812
`meeting`, direct route	clean	—	—	0.0	0 / 106 / 1812
`moving`, Event-Calculus route, no initiation clause on any fold	clean	—	—	—	0 / 0 / 3136
`moving`, direct route, canonical `close_34`	clean	0.5334	0.3817	0.4450	1197 / 1047 / 1939
`moving`, direct route, earlier run under `close_25`	clean	—	—	0.4868	1295 / 890 / 1841

Table 13: Design-cost isolations. Verification splits the cold compile on probabilistic reachability chains of length n. Residency forces a device→host→device round-trip at the neural–symbolic boundary, swept over 2 to 512 handoffs per step on the batched addition path.

Setting	Measured
Verification cost	
chain n=5	CDCL verify 531 ms
chain n=40	CDCL verify 2,633 ms
Residency benefit	
per handoff	round-trip of roughly 61–176 μs
32 handoffs/step	9.2% of the step
128 handoffs/step	16.1% of the step; this is the standard batch-64 MNIST-addition step, 9.6 ms of 59.3 ms
512 handoffs/step	holds near 15.5% of the step

boundary, the transfer a CPU-reasoning hybrid pays every step, sweeping the number of neural→symbolic handoffs per step. Below a handful of handoffs the round-trip cost is lost in run-to-run variation. From a few dozen upward it is a visible share of the step, because the batched circuit compute grows sublinearly with the batch while the transfer grows linearly with the handoff count.

On a single query the transfer is therefore negligible (2.7% at two handoffs); at a realistic training batch it is a ≈16% tax that xlog does not pay, distinct from and additive to the GPU-compute and circuit-caching advantages. The forced per-buffer copy is an upper bound, since a hybrid could coalesce transfers. The larger transfer-dominated regime, large-state Datalog fixpoints where whole relations would cross the bus every iteration (Section 1), lies outside what this isolation measures (Section 12).

10.7 Index reuse and the chain scorer

This pair of ablations asks what two of the runtime optimizations of Section 4 save on the repeated-session workloads they target, each against the path it replaces. The persistent hash-index manager retains built join indices across evaluations, keyed on relation generation, schema signature and device ordinal, under a byte-budgeted LRU policy: a session whose relations change little between calls reuses an index instead of rebuilding it, and a generation change invalidates the entry instead of serving a stale one. The profile-gated shared-memory chain scorer tiles the left relation through shared memory when scoring chain-shaped candidate bodies during exact rule induction, and is gated on a candidate-row threshold below which the baseline scorer runs unchanged.

Both target the same waste, work repeated across calls that the previous call already did, and both are transparent to the program. Common-subexpression elimination and adaptive re-optimization are held to output equivalence with the unoptimized plan.

Table 14 gives the medians. Two things belong with them. The index fixture was re-measured in release after an earlier version of its artifact recorded a build flag the run had not used; the same pod then ran both profiles back to back, and release is the *higher* ratio, 8.33× against 7.72× in debug. The reason is that the two arms sit on different sides of the bus: the index-rebuilding arm is GPU-bound and barely moves between profiles (15.83 to 15.78 ms), while the cached arm is host-side and does get faster (2.05 to 1.89 ms), so optimizing the build widens the gap. And the chain scorer's ratio rose from 5.58× to 7.20× partly because its baseline arm got slower on this card, 27.51 to 52.42 ms, so that gain is not a like-for-like improvement over the earlier record. The index fixture is build-heavy by construction, so that building the index dominates the join, and neither arm records a host transfer. The chain scorer's two arms do not overlap on the chain-hot fixture; on a small input below its gate threshold, where the baseline path runs instead, the two medians differ by 2.0% and the arms' ranges overlap. Both are point medians on single synthetic fixtures, and the scorer's coverage output is identical to the baseline's on both. The scorer's artifact now also records the host-transfer count per arm rather than asserting a fixed one: what the claim rests on is that the shared-memory arm adds none relative to baseline, and the measured difference is zero.

Table 14: Runtime-optimization ablations, each a median over the timed runs of one synthetic fixture.

Fixture	Baseline median → optimized median
Persistent index reuse: repeated-session semi-join, eight left rows probing eight million right rows	15.78 ms → 1.89 ms, a 8.33× gain over 9 timed runs after 12 warm-up runs
Shared-memory chain scorer: chain-hot body search, 768 candidate rows against a gate threshold of 256	52.42 ms (52.41–52.97) → 7.28 ms (7.28–7.29), a 7.20× gain over 12 timed runs after 3 warm-up runs

10.8 Capability coverage across systems

Table 15 positions xlog qualitatively against representative systems; each addresses a subset of the design space, and xlog's contribution is exposing the supported routes through one typed language and provider-owned CUDA runtime services. This table covers capability: the quantitative head-to-head results against Scallop, ProbLog2 and Soufflé are in Section 10.4.

The rule-induction row covers two paths of different maturity: the trainable-rule-body path is carried onto an external benchmark in Section 7 and tabulated in

Section 10.5, while the differentiable-ILP path is not exercised there at all (Section 6). The table's "yes" claims capability, not maturity (Section 12).

10.9 Log-evidence, rule unions, and exact constraint solving

The capabilities below round out the runtime surface. Each is described for what it does; no timing is claimed for any of them.

Exact log-evidence and the CNF-variable-to-fact map. An exact evaluation now returns the log partition function of the weighted formula, $\log Z$, alongside the per-query probabilities, so a program's evidence mass is read off directly instead of being reconstructed from marginals; it surfaces as `log_z_e` on the Python result. Paired with it, `prob_var_map()` reports which probabilistic fact each CNF variable stands for (an ordinary fact, one Bernoulli decision of an annotated disjunction's chain, or padding) so the per-variable gradient buffers can be attributed back to program facts and not to opaque indices. Its length is the encoder's variable *capacity*, not a variable count, and it is refused with a typed error where there is no CNF encoding to report: Monte Carlo programs, and exact programs taking the GPU fast path that skips the encoding.

Same-head rule unions in one multiway pass. A predicate head defined by many rules used to fold its per-rule contributions into the head relation one union at a time, re-sorting the growing accumulator once per rule and going quadratic in the rule count, a real cost at corpus scale where one head can carry thousands of rules. Contributions are now merged with a single multiway union per head per iteration. The union count therefore no longer scales with the same-head rule count (eight same-head rules record one union, not eight) while the derived rows stay byte-identical to the unbatched result.

An exact stage beyond enumeration capacity. Constraint solving over several interacting constraints enumerates a component completely where it is small enough. Long chain-shaped components that exceed that capacity take a memoized dynamic-programming stage on the GPU instead, which computes the same totals exactly instead of approximating them. A component wider than the configured limit is refused with a typed error instead of served by an approximation, and the solver's buffers reach Python through the same zero-copy DLPack [6] views as the rest of the runtime (Section 6.3).

11 Related Work

xlog draws on five lines of research: neurosymbolic programming, GPU-accelerated Datalog, probabilistic logic programming, GPU SAT, and differentiable ILP. Its contribution is to expose those capabilities through one typed language and shared CUDA services while retaining route-specific execution and differentiability contracts.

11.1 Neurosymbolic programming

DeepProbLog [1] replaces selected probabilistic facts with neural-network outputs, enabling end-to-end gradient training of neural predicates inside a probabilistic-logic framework. NeurASP [2] integrates network outputs as probability annotations on answer-set programs. Both perform symbolic inference on the CPU and shuttle gradients across the bus. Scallop [29] provides a differentiable Datalog with provenance-semiring reasoning [13] and a relational symbolic representation, and is the closest language-level analogue to xlog. Its reasoning core, however, is CPU-based.

Lobster [30] is the most directly comparable system: it GPU-accelerates neurosymbolic programs in the Scallop style, and shares xlog's premise that the symbolic data plane belongs on the GPU. Against it, xlog is complementary along three axes. It includes exact knowledge compilation and epistemic reasoning in addition to provenance-based differentiable Datalog. It certifies the final smoothed circuit's logical equivalence to its back-end source formula using GPU CDCL and on-device proof checking. And it reports route-specific residency with byte-level transfer accounting. No head-to-head performance comparison with Lobster is drawn here (Section 12).

Differentiable-SAT and logic-as-loss methods (SATNet [31], semantic loss [32], and Logic Tensor Networks [33]) instead couple neural models to constraints through relaxed or fuzzy surrogates. By contrast, xlog couples through *exact* weighted model counting, trading their broader applicability for exact gradients and the on-device verifiability of Section 5.

11.2 GPU-accelerated Datalog and joins

GPUlog [3] ported bottom-up fixed-point computation to CUDA using HISA indexing, and reported up to 45× speedups over CPU Datalog engines. VFLog [4] advanced this with a vertically-fused, column-oriented kernel design that avoids intermediate materialization, reporting up to 200× over CPU column engines; mnmgDatalog [34] extends GPU Datalog to multi-node, multi-GPU clusters. Most relevant to xlog's join path, Sun et al. [35] scale worst-case-optimal joins to GPUs.

All of them target deterministic Datalog only: none supports probabilistic semantics, weighted model counting, or gradient computation over derived facts. Like them, xlog uses a columnar, kernel-fused execution model, and it also promotes eligible cyclic and multiway rules to a worst-case-optimal join path [9–11]. But that engine is one component of a substrate that extends through circuit-based inference to neural-symbolic gradient propagation in a single address space.

Table 15: Capability comparison across neurosymbolic and GPU logic systems.

Dimension	DeepProbLog	Scallop	Lobster	GPUlog	xlog
Symbolic execution	CPU (Prolog)	CPU (Datalog)	GPU (Datalog)	GPU (Datalog)	GPU (semi-naive)
Probabilistic inference	Exact (CPU)	Provenance (CPU)	Provenance (GPU)	None	Exact d-DNNF + MC (GPU)
Circuit verification	None	None	None	—	On-GPU CDCL
Neural integration	Prolog bridge	Differentiable	Differentiable (GPU)	None	Zero-copy, fused
Zero-copy ML interop	No	Partial	Partial	No	Yes
Rule structure induction	No	No	No	No	Yes (beta): trainable bodies, dILP
Epistemic reasoning	No	No	No	No	Yes (finite fragment)

11.3 Probabilistic logic programming

ProbLog2 [12] established the modern pipeline for exact probabilistic inference: ground the program, encode provenance as a propositional formula, compile to a tractable target (d-DNNF [15] or SDD [36]), and evaluate weighted model counts [19]. The compilers (D4 [16], c2d [37]) run as host-side subprocesses, and recent work makes such compilation *certified* [20]. xlog adopts the same compile-once/evaluate-many model with host provenance construction followed by upload and CUDA-backed Tseitin encoding, D4 compilation, equivalence verification, and forward/backward evaluation. The host orchestrates the device stages and reads bounded solver control state; device circuit and gradient buffers share an address space with the neural and deterministic data planes.

11.4 GPU SAT

ParaFROST [5] parallelizes CDCL clause-database simplification on the GPU while retaining sequential conflict-driven search on the CPU. FastFourierSAT [38] reformulates SAT as continuous optimization and applies GPU local search, an incomplete solver for satisfiable instances. Both are standalone competition solvers. In xlog the GPU CDCL module is instead a verification component inside knowledge compilation (Section 5), proving circuit–formula equivalence via two UNSAT proofs and providing the certificates the probabilistic and epistemic backends rely on.

11.5 Differentiable ILP

Differentiable ILP [39] casts first-order rule induction as differentiable optimization, scoring candidate rules by soft forward-chaining. Dense materialization over the rule space scales cubically in the number of constants, and implementations remain CPU-based. The dILP subsystem of xlog (Section 6) replaces dense materialization with sparse GPU bitmasks and on-device scatter–gather credit assignment, avoiding the cubic blowup while preserving differentiability.

11.6 Logic programming languages

xlog inherits ideas from a long line of logic languages. Prolog [40] offers unification and metaprogramming on a CPU interpreter; Mercury [41] introduced strong typing and modes, compiling to native code. Soufflé [42] is a typed Datalog compiling to C++ for program analysis, and clingo [43] provides answer-set semantics on CPU grounders and solvers. From these xlog adopts the typed-predicate discipline while unifying typed Datalog, probabilistic facts, neural predicates, and epistemic operators in one language whose supported routes use CUDA kernels and provider-owned runtime services.

12 Limitations and Future Work

12.1 Current limitations

NVIDIA GPU required. The execution kernels are written in CUDA C; there is no OpenCL, Metal, or ROCm backend. The persistent-index manager, recorded-launch discipline, and in-kernel hash-consing use CUDA-specific primitives such as cooperative groups, warp-level operations, and unified virtual addressing. Users without a supported NVIDIA GPU cannot run xlog.

Device memory and fixed capacities are bounded. Relations, indices, circuit buffers, and Monte Carlo sample arrays are allocated on-device; there is no out-of-core spilling. An allocation that exceeds the configured byte budget fails with `RESOURCE_EXHAUSTED`. A row, tuple, sample, domain, or other fixed-count limit instead fails with `CAPACITY_EXCEEDED`, naming the exhausted unit. A bounded recursive or Monte Carlo fixpoint that reaches its iteration limit without convergence fails with `CONVERGENCE_FAILURE`; partial counters do not turn it into a result.

Maturity of the rule-learning surfaces. The differentiable-ILP subsystem (Section 6) is beta: the search space is restricted to definite programs, convergence is sensitive to learning-rate schedules, and its API may change. The trainable-rule-body surface of Section 6.4 (mixed neural/symbolic bodies, existential joins, and joint multi-rule mixtures) is at the same maturity. It is carried onto a public benchmark under pre-registered gates and cross-validation in Section 7, but how its accuracy and training cost compare with dedicated relational learners on standard benchmarks is an open question.

Coverage of the epistemic runtime. The epistemic runtime (Section 9) executes its accepted data-plane work on the GPU, but the broader solver portfolio (MaxSAT/portfolio services) and the end-to-end probabilistic–epistemic evidence path are still being broadened. The negated-modal WFS route and Gelfond-1991 compatibility refinement are GPU-backed but host-orchestrated. Positive modal dependency cycles have defined execution: FAEEL computes a founded least fixpoint, including an empty extension for an unseeded cycle, and supported Gelfond-1991 positive `possible` cycles compute the greatest compatible set of concrete tuples from an upper bound and frozen snapshots. The shared recursion-depth setting bounds both descending compatibility refinements and WFS iterations.

Fail-closed boundaries. Raw lowering of modal literals, recursive epistemic programs that also contain modal integrity constraints, WFS shapes outside the supported negated-modal plan, unbounded tuple-key forms, and recursive negation or aggregation within a Gelfond-1991 compatibility component all remain fail-closed. The Python batch-query helpers are typed but do not yet accept floating-point uploads, a convenience-API gap and not a core execution limit.

Partial head-to-head coverage. Section 10.4 reports controlled comparisons on identical programs and data against one external engine per reasoning mode: Scallop (neural-symbolic), ProbLog2 (probabilistic), and Soufflé (deterministic). Coverage is still partial. Lobster, the most directly comparable GPU-neurosymbolic system, and the GPU Datalog engines GPUlog and VFLog are not among the baselines; DeepProbLog was not run under a matched (non-timeout) protocol; and each comparison rests on a single GPU configuration rather than a cross-hardware sweep. That last limitation is not hypothetical: measuring the MNIST comparison on two L40S pods with the same wheel, the same seeds and different CPU quotas moved Scallop's steady epoch from 31.79 s at 13.6 cores to 54.99 s at 27.2, so the epoch ratio between the two systems changed by a factor of 1.8 with neither engine touched. The pods differed in driver revision and host as well as in quota, so the quota is the candidate explanation rather than the isolated cause; the 27.2-core measurement is distributed as the companion file beside the MNIST artifact. That is why Section 10.4 reports accuracy for that comparison and no timing. A per-cell ratio measured on one host is a property of the pair *and* the host; broader multi-system, multi-hardware comparison remains future work.

Rule-induction evidence. The evidence of Section 7 is thinnest exactly where it is leak-free: the deduplicated, scene-family-grouped corpus carries only eleven observable `meeting` initiations. The direct route commits a clause on nine of ten folds; the Event-Calculus route commits one on one fold and abstains on the other nine. Neither transfers: their pooled true-positive counts are zero, with 106 and 120 false positives respectively. This is failed transfer of selected clauses, alongside abstention, rather than an empty theory on every fold; it establishes neither general learnability nor impossibility. The protocol-matched numbers are computed instead on the dump distributed with the published Event-Calculus learners, which carries a measured duplication defect; we report there because it is the regime the published figures share, and every figure computed on it inherits that caveat. No ordering against OLED, WOLED-ASP or the hand-crafted rule set follows from Table 11, and none is claimed: the five protocol deltas printed with that table are real, unquantified, and do not act in the same direction. The pre-registered run learns no termination clause; the adaptive vocabulary iteration learns termination clauses on seven of ten folds, but a full termination theory and a broader body language remain unestablished.

Two figures of different status. The headline figure of Section 7 is fully pre-registered: gates, seeds and folds were fixed before the run, and that run was executed once. The second-iteration figure differs from it in exactly one respect, its Event-Calculus vocabulary having been chosen after the first run's failure mode was known on the same folds, which makes it one disclosed adaptive iteration and not a pre-registered result; it is reported only with that label and is never the row we place beside a published number.

Maritime scope. The maritime runs of Section 8 remove the evidence-starvation bound, but they carry bounds of their own. Their supervision is synthetic: the gold labels are a hand-crafted rule's output over the same streams the vocabulary is derived from, a regime the published maritime figures share. Every number there is therefore a rule-reconstruction figure under an incomplete vocabulary, not a detection figure, and none of those runs exercises the GPU substrate. Their temporal grid is the critical-point grid of the RTEC exports, not the dense grid of the published figure, so no maritime number of ours stands beside the published 0.98 and no pass time beside the published training times (Section 8.4). And the single-pass runs are online in the weights only, their body pool and permutation-null gate being computed on the training side: they demonstrate single-pass weight learning and not online structure revision, which remains future work, and the zero degradation they measure rests on a single body sitting above the decision threshold in

every regime, on a 0.007 margin; it is a property of this pool, corpus and threshold, not a robustness result.

12.2 Future work

Out-of-core execution. A streaming mode would partition relations across host and device, paging tiles through GPU memory in fixpoint-iteration order, removing the single-GPU-memory ceiling at the cost of added transfers.

Multi-GPU partitioned evaluation. Inspired by mnmgDatalog's [34] radix-hash partitioning and GPU-aware all-to-all shuffle, a multi-GPU backend would distribute relations across devices and coordinate joins over NVLink or PCIe.

Deeper epistemic residency. The remaining epistemic work is stronger residency and coverage: a device-resident, no-host-interaction well-founded path, broader non-finite tuple-key forms, and semantic forms beyond the current finite accepted surface.

13 Conclusion

xlog is a CUDA-native logic programming engine with one typed language and provider-owned runtime services for deterministic, probabilistic, epistemic, and neural-symbolic use. A neural network is an ordinary predicate; exact query gradients flow through the probabilistic knowledge-compilation route, while the other routes reuse typed device-buffer interfaces without claiming identical differentiability. The final smoothed probabilistic circuit is certified against its back-end source formula before it is cached or evaluated. Solver proof traces are checked on the GPU, and the host observes bounded status and error scalars.

Residency claims are likewise route-specific. Ordinary relational and exact execution are host-orchestrated over device data planes. Eligible recursion and resident Monte Carlo record zero tracked transfers within their measured cores. The recursive graph then returns one pinned terminal receipt; Monte Carlo reads three postflight metadata words and leaves result reads separate. Resource, capacity, and convergence failures must be interpreted through each route's actual diagnostic rather than treated as a completed result.

The measurements describe a system that pays off where the symbolic workload is large and repeated: reusing a verified circuit across training iterations, and avoiding the intermediate blowup of binary-join plans on skewed cyclic queries. Against external engines the picture is mixed: bounded-memory join evaluation improves with input size on the measured skewed graphs, MNIST-addition accuracy is comparable to Scallop's without a per-epoch timing claim, and small exact-inference programs run substantially slower than in ProbLog2.

What they do not give is a general ranking of xlog against other systems. Each comparison covers one engine per reasoning mode on one class of program, the rule-induction results rest on two public corpora, and the epistemic and rule-learning surfaces are young. The evidence supports the architecture, not a claim of universal superiority.

The direction is the one the architecture already implies: out-of-core and multi-GPU execution to lift the memory ceiling, deeper residency for the epistemic routes, and structure learning that runs on the stream rather than over it.

References

[1] Robin Manhaeve et al. "DeepProbLog: Neural Probabilistic Logic Programming". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[2] Zhun Yang, Adam Ishay, and Joohyung Lee. "NeurASP: Embracing Neural Networks into Answer Set Programming". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*. 2020.

[3] Yihao Sun et al. "Optimizing Datalog for the GPU". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2025. DOI: 10.1145/3669940.3707274.

[4] Yihao Sun et al. "Column-Oriented Datalog on the GPU". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 15177–15185. DOI: 10.1609/aaai.v39i14.33665.

[5] Muhammad Osama, Anton Wijs, and Armin Biere. "SAT Solving with GPU Accelerated Inprocessing". In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12651. Lecture Notes in Computer Science. Springer, 2021, pp. 133–151. DOI: 10.1007/978-3-030-72016-2_8.

[6] *DLPack: Open In Memory Tensor Structure*. https://github.com/dmlc/dlpack. Accessed 2026.

[7] *cudarc: Safe Rust Wrapper around CUDA*. https://github.com/coreylowman/cudarc. Accessed 2026.

[8] *PyO3: Rust Bindings for Python*. https://pyo3.rs. Accessed 2026.

[9] Hung Q. Ngo et al. "Worst-case Optimal Join Algorithms". In: *Journal of the ACM* 65.3 (2018), 16:1–16:40.

[10] Todd L. Veldhuizen. "Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm". In: *International Conference on Database Theory (ICDT)*. 2014, pp. 96–106.

[11] Yisu Remy Wang, Max Willsey, and Dan Suciu. "Free Join: Unifying Worst-Case Optimal and Traditional Joins". In: *Proceedings of the ACM on Management of Data (SIGMOD)* 1.2 (2023), 150:1–150:23.

[12] Daan Fierens et al. "Inference and Learning in Probabilistic Logic Programs using Weighted Boolean Formulas". In: *Theory and Practice of Logic Programming* 15.3 (2015), pp. 358–401.

[13] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. "Provenance Semirings". In: *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. 2007, pp. 31–40.

[14] Grigori S. Tseitin. "On the Complexity of Derivation in Propositional Calculus". In: *Studies in Constructive Mathematics and Mathematical Logic* (1968).

[15] Adnan Darwiche. "Decomposable Negation Normal Form". In: *Journal of the ACM* 48.4 (2001), pp. 608–647.

[16] Jean-Marie Lagniez and Pierre Marquis. "An Improved Decision-DNNF Compiler". In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI)*. 2017.

[17] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference (DAC)*. 2001, pp. 530–535.

[18] Adnan Darwiche and Pierre Marquis. "A Knowledge Compilation Map". In: *Journal of Artificial Intelligence Research* 17 (2002), pp. 229–264.

[19] Mark Chavira and Adnan Darwiche. "On Probabilistic Inference by Weighted Model Counting". In: *Artificial Intelligence* 172.6-7 (2008), pp. 772–799.

[20] Florent Capelli. "Knowledge Compilation Languages as Proof Systems". In: *Theory and Applications of Satisfiability Testing (SAT)*. Vol. 11628. Lecture Notes in Computer Science. Springer, 2019, pp. 90–99.

[21] *Apache Arrow: Cross-Language Development Platform for In-Memory Data*. https://arrow.apache.org. Accessed 2026.

[22] Nikos Katzouris, Alexander Artikis, and Georgios Paliouras. "Online Learning of Event Definitions". In: *Theory and Practice of Logic Programming* (2016). arXiv: 1608.00100.

[23] Nikos Katzouris, Georgios Paliouras, and Alexander Artikis. "Online Learning Probabilistic Event Calculus Theories in Answer Set Programming". In: *Theory and Practice of Logic Programming* 23.2 (2023), pp. 362–386. DOI: 10.1017/S1471068421000107. arXiv: 2104.00158.

[24] Cyril Ray et al. "Heterogeneous Integrated Dataset for Maritime Intelligence, Surveillance, and Reconnaissance". In: *Data in Brief* 25 (2019), p. 104141. DOI: 10.1016/j.dib.2019.104141.

[25] Manolis Pitsikalis et al. "Composite Event Recognition for Maritime Monitoring". In: *Proceedings of the 13th ACM International Conference on Distributed and Event-based Systems (DEBS)*. 2019, pp. 163–174. DOI: 10.1145/3328905.3329762.

[26] Alexander Artikis, Marek Sergot, and Georgios Paliouras. "An Event Calculus for Event Recognition". In: *IEEE Transactions on Knowledge and Data Engineering* 27.4 (2015), pp. 895–908. DOI: 10.1109/TKDE.2014.2356476.

[27] Lars Ole Andersen. "Program Analysis and Specialization for the C Programming Language". PhD thesis. University of Copenhagen, 1994.

[28] Antonio Flores-Montoya and Eric Schulte. "Datalog Disassembly". In: *USENIX Security Symposium*. 2020, pp. 1075–1092.

[29] Ziyang Li, Jiani Huang, and Mayur Naik. "Scallop: A Language for Neurosymbolic Programming". In: *Proceedings of the ACM on Programming Languages (PLDI)* 7 (2023), 166:1–166:25.

[30] Paul Biberstein et al. "Lobster: A GPU-Accelerated Framework for Neurosymbolic Programming". In: *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. arXiv:2503.21937. 2026.

[31] Po-Wei Wang et al. "SATNet: Bridging Deep Learning and Logical Reasoning Using a Differentiable Satisfiability Solver". In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 6545–6554.

[32] Jingyi Xu et al. "A Semantic Loss Function for Deep Learning with Symbolic Knowledge". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, pp. 5502–5511.

[33] Samy Badreddine et al. "Logic Tensor Networks". In: *Artificial Intelligence* 303 (2022), p. 103649.

[34] Ahmedur Rahman Shovon et al. "Multi-Node Multi-GPU Datalog". In: *Proceedings of the 39th ACM International Conference on Supercomputing (ICS)*. 2025. DOI: 10.1145/3721145.3730431.

[35] Yihao Sun et al. *Scaling Worst-Case Optimal Datalog to GPUs*. arXiv:2604.20073. 2026.

[36] Adnan Darwiche. "SDD: A New Canonical Representation of Propositional Knowledge Bases". In: *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, pp. 819–826.

[37] Adnan Darwiche. “New Advances in Compiling CNF into Decomposable Negation Normal Form”. In: *Proceedings of the 16th European Conference on Artificial Intelligence (ECAI)*. 2004.

[38] Yunuo Cen, Zhiwei Zhang, and Xuanyao Fong. *Massively Parallel Continuous Local Search for Hybrid SAT Solving on GPUs*. arXiv:2308.15020. 2023.

[39] Richard Evans and Edward Grefenstette. “Learning Explanatory Rules from Noisy Data”. In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 1–64.

[40] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[41] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. “The Execution Algorithm of Mercury, an Efficient Purely Declarative Logic Programming Language”. In: *Journal of Logic Programming* 29.1-3 (1996), pp. 17–64.

[42] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. “Soufflé: On Synthesis of Program Analyzers”. In: *Computer Aided Verification (CAV)*. Springer, 2016, pp. 422–430.

[43] Martin Gebser et al. “Multi-shot ASP Solving with clingo”. In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.